

MBA EM **DATA SCIENCE E ANALYTICS**

Cloud Computing II

Professor Rodrigo Vale

Larissa Dos Santos Magistrali 867.843.400-78

MBAUSP
ESALQ

A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.

Proibida a reprodução, total ou parcial, sem autorização.

Lei nº 9610/98

Larissa Dos Santos Magistral 867.843.400-78

MBAUSP
ESALQ

O que são dados?

Dados são fatos e números brutos, desorganizados e não processados. Pense neles como uma coleção de números, palavras, medições ou observações individuais.

Por si só, dados brutos carecem de contexto e significado. São o bloco de construção fundamental a partir do qual o valor pode ser derivado.

Exemplos:

- Uma lista de temperaturas diárias: 25, 26, 28, 24, 29
- Valores de compra do cliente: \$ 12,50, \$ 45,21, \$ 8,99, \$ 102,00
- Cliques no site: Clique, Clique, Não Clique, Clique

Dos dados ao insight

Os dados se tornam valiosos quando são processados e analisados. Essa jornada tem três etapas principais:

Dados brutos: o ponto de partida.

- Exemplo: 120 itens azuis vendidos, 150 itens vermelhos vendidos, 90 itens verdes vendidos.

Informação: Dados que são processados, organizados e estruturados para fornecer contexto.

- Exemplo: "Os itens vermelhos foram os produtos mais vendidos neste mês."

Insight: O profundo entendimento obtido pela análise das informações, que pode orientar ações e decisões.

- Exemplo: "Nossa campanha de marketing para itens vermelhos foi altamente eficaz. Devemos aplicar estratégias semelhantes aos nossos itens azuis e verdes para aumentar suas vendas."

Por que os dados são chamados de "o novo petróleo"?

A frase "Dados são o novo petróleo" destaca sua importância econômica moderna.

- **Potencial não refinado:** Assim como o petróleo bruto, os dados brutos são valiosos, mas não úteis em seu estado natural. Eles precisam ser processados e refinados.
- **O processo de refinamento:** os dados são "refinados" por meio da análise, o que os transforma em informações valiosas e insights acionáveis.
- **Impulsionando o Futuro:** Assim como o petróleo impulsionou a Revolução Industrial, os dados impulsionam a era digital. Eles permitem que as empresas tomem decisões mais inteligentes, entendam os clientes, prevejam tendências e criem produtos e serviços inovadores.

As empresas mais bem-sucedidas hoje são aquelas que dominam a arte de refinar dados em insights que impulsionam seus negócios.

Os quatro tipos de análise:

Descritivo: O que aconteceu? (por exemplo, revisão do relatório de vendas do último trimestre).

Diagnóstico: Por que isso aconteceu? (por exemplo, detalhando para ver em qual região as vendas caíram).

Preditivo: O que acontecerá? (por exemplo, previsão da demanda futura).

Prescritiva: O que devemos fazer? (por exemplo, recomendar uma estratégia de marketing).

Análise Descritiva

O que aconteceu?

A análise descritiva é a base da análise de dados. Ela envolve a sumarização de dados históricos para compreender o desempenho e os eventos passados. Esta é a forma mais comum de análise utilizada pelas empresas.

- Objetivo: Obter uma imagem clara do que já ocorreu.
- Métodos: Agregação de dados, mineração de dados e relatórios.
- Exemplo: revisar um relatório de vendas do trimestre anterior para ver a receita total, as unidades vendidas e os produtos de melhor desempenho.
- Palavras-chave: Relatório, resumo, painel, visão geral.

Análise de Diagnóstico

Por que isso aconteceu?

Depois de saber o que aconteceu, o próximo passo é entender o porquê. A análise diagnóstica se aprofunda nos dados para encontrar as causas-raízes dos resultados.

- **Objetivo:** Identificar os fatores e as razões por trás de um determinado evento ou tendência.
- **Métodos:** Detalhamento, descoberta de dados, correlações e análise de causa raiz.
- **Exemplo:** depois de observar uma queda nas vendas (descritiva), você analisa os dados detalhadamente e descobre que a queda foi causada principalmente por uma única região de baixo desempenho.
- **Palavras-chave:** Investigar, aprofundar, causa raiz, determinar o porquê.

Análise preditiva

O que vai acontecer?

A análise preditiva utiliza dados históricos, modelos estatísticos e técnicas de aprendizado de máquina para prever eventos futuros. Trata-se de fazer suposições fundamentadas sobre o que está por vir.

- Objetivo: Antecipar tendências, oportunidades ou riscos futuros.
- Métodos: Modelagem estatística, aprendizado de máquina e previsão.
- Exemplo: usar dados de vendas anteriores e tendências de mercado para prever a demanda de um produto na próxima temporada de festas de fim de ano.
- Palavras-chave: Previsão, predizer, futuro, estimativa, tendência.

Análise prescritiva

O que devemos fazer?

A análise prescritiva é o estágio mais avançado. Ela vai além da previsão para recomendar ações específicas a serem tomadas para alcançar um resultado desejado ou mitigar um risco futuro.

- **Objetivo:** Fornecer recomendações claras e orientar a tomada de decisões.
- **Métodos:** Algoritmos de otimização, simulação e inteligência artificial.
- **Exemplo:** Com base em uma previsão de vendas baixas em uma região, o sistema recomenda uma estratégia de marketing específica, estima o ROI potencial e sugere níveis ideais de estoque.
- **Palavras-chave:** Recomendar, otimizar, ação, e se, estratégia.

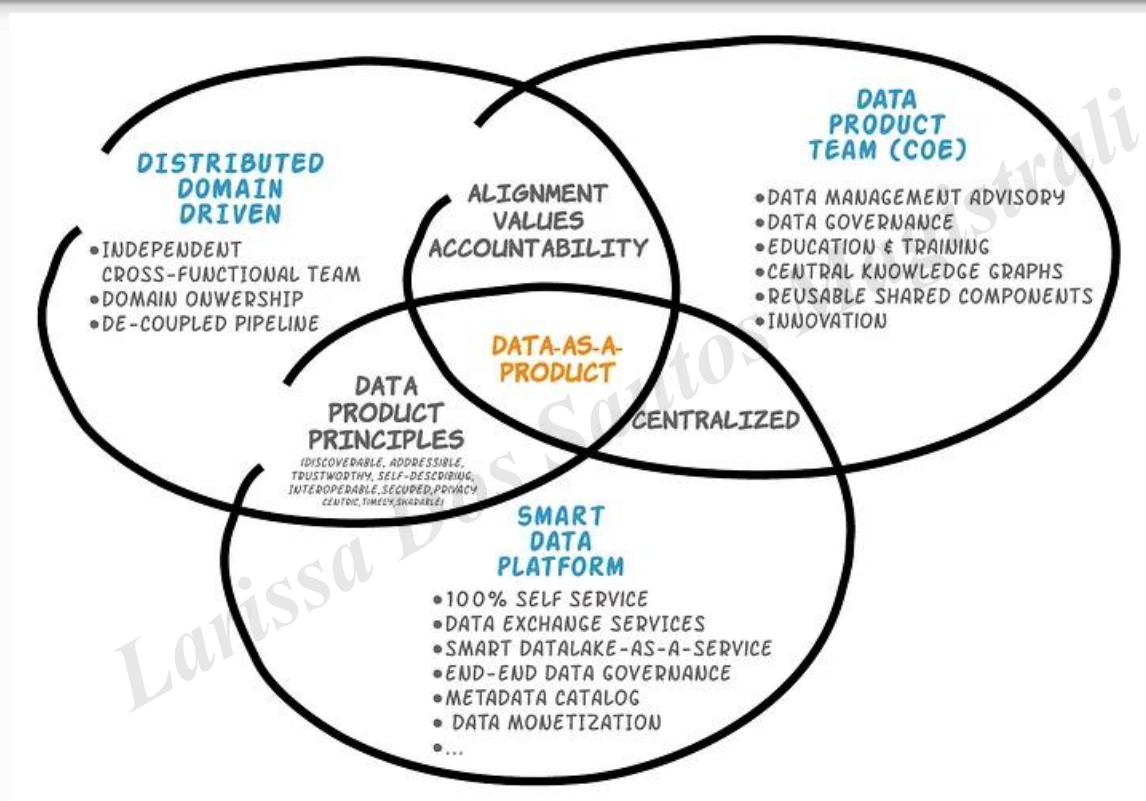
Dados como um produto (DaaP)

A ideia central: trate os dados como um produto acabado, não como um subproduto bruto.

A mudança de mentalidade

De: A Visão Tradicional	Para: A Visão do Produto
Os dados são um escape/subproduto	Os dados são um ativo essencial
Foco em pipelines e tecnologia	Foco nas necessidades e no valor do consumidor
Equipes centralizadas e baseadas em projetos	Equipes de produtos descentralizadas e duráveis
A qualidade dos dados é uma reflexão tardia	A qualidade dos dados é uma característica principal
A propriedade é ambígua ou temporária	A propriedade é clara e de longo prazo

Elementos-chave de uma abordagem DaaP



<https://medium.com/google-cloud/implementing-data-as-a-product-daap-using-distributed-data-architecture-and-smart-data-platform-on-c2fcd64c67d5>

O que torna os dados um "produto"?

Um conjunto de dados se torna um produto quando:

- **Descobrível:** os consumidores podem encontrá-lo facilmente (por exemplo, em um catálogo de dados).
- **Endereçável:** Possui uma localização permanente, única e de fácil acesso.
- **Confiável:** é confiável, preciso e tem Objetivos de Nível de Serviço (SLOs) claros para qualidade e atualização.
- **Compreensível:** é autodescritivo, com metadados, esquema e documentação claros.
- **Seguro:** é regido por políticas claras de propriedade e acesso.
- **Valioso por si só:** foi projetado para solucionar diretamente uma necessidade comercial de seus consumidores.

Por que é importante: os benefícios

Maior ROI: maximiza o valor derivado dos ativos de dados.

Maior agilidade: capacita as equipes a se auto atender, acelerando a análise e a inovação.

Melhoria na qualidade, confiança e tomada de decisões: promove a responsabilização, resultando em dados mais confiáveis e decisões mais acertadas. Dados acessíveis e de alta qualidade permitem uma tomada de decisões mais eficiente e baseada em dados em toda a organização.

Redução de Custos/Eficiência Operacional: Elimina pipelines de dados redundantes e retrabalho. Os dados podem ser usados para identificar áreas de melhoria e otimizar processos.

Desbloqueando novos fluxos de receita: as organizações podem criar novos produtos e serviços com base em seus dados ou vender dados diretamente para outras partes.

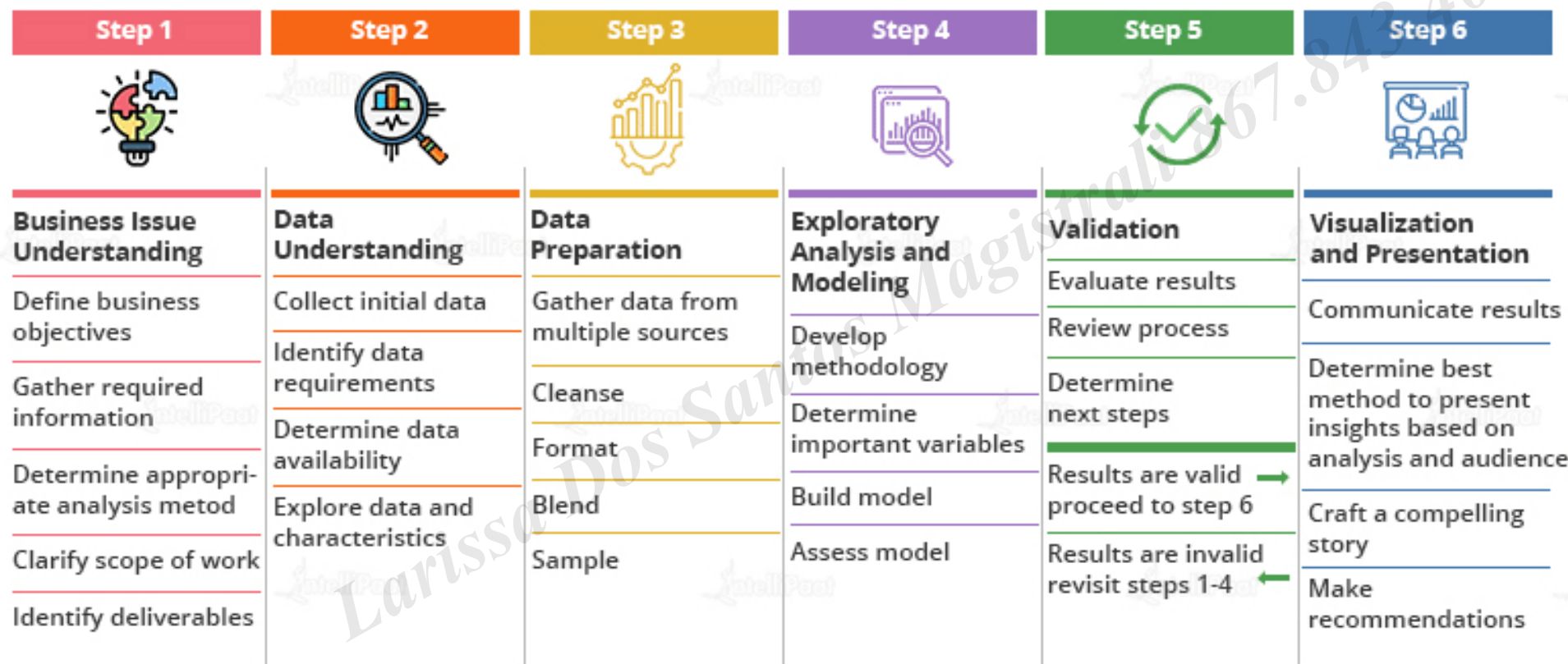
O ciclo de vida da análise de dados

Larissa Dos Santos Magistrali 867.843.400-78

Ciclo de Vida do Projeto de Análise de Dados

LIFE CYCLE OF DATA ANALYSIS PROJECT

Based on CRISP-DM Methodology



<https://intellipaat.com/blog/tutorial/data-analytics-tutorial/data-analytics-lifecycle/>

Ciclo de vida da análise de dados

Coletar

O que é: A base da análise. Esta fase envolve a coleta de dados brutos de diversas fontes.

As fontes podem incluir:

pesquisas, sensores, mídias sociais, bancos de dados, APIs e web scraping.

Objetivo: Obter as informações necessárias para responder a uma pergunta específica ou resolver um problema.

Processo

O que é: Limpeza e transformação de dados brutos em um formato utilizável. Dados brutos costumam ser confusos, incompletos ou inconsistentes.

Principais atividades:

- Lidando com valores ausentes.
- Corrigindo erros.
- Removendo duplicatas.
- Estruturação dos dados para análise.

Objetivo: Garantir a qualidade e a confiabilidade dos dados.

Armazenar

O que é: Armazenamento seguro dos dados processados para uso atual e futuro.

Métodos:

Armazenamento de dados em bancos de dados (como SQL ou NoSQL), data warehouses ou data lakes.

Objetivo: Fornecer acesso confiável e de longo prazo aos dados de maneira organizada e eficiente.

Analisar

O que é: O núcleo do ciclo de vida, onde os insights são descobertos. Isso envolve examinar, modelar e interpretar os dados.

Técnicas: Análise estatística, algoritmos de aprendizado de máquina, visualização de dados e análise exploratória de dados.

Objetivo: Encontrar padrões, tendências e correlações que respondam à pergunta inicial.

Ativar

O que é: Colocar os insights em ação. Esta fase preenche a lacuna entre a análise e o valor comercial.

As ações podem incluir:

- Tomando decisões baseadas em dados.
- Implementação de mudanças em processos de negócios.
- Desenvolver novos produtos ou serviços.
- Personalizando experiências do cliente.

Objetivo: Traduzir descobertas analíticas em resultados tangíveis.

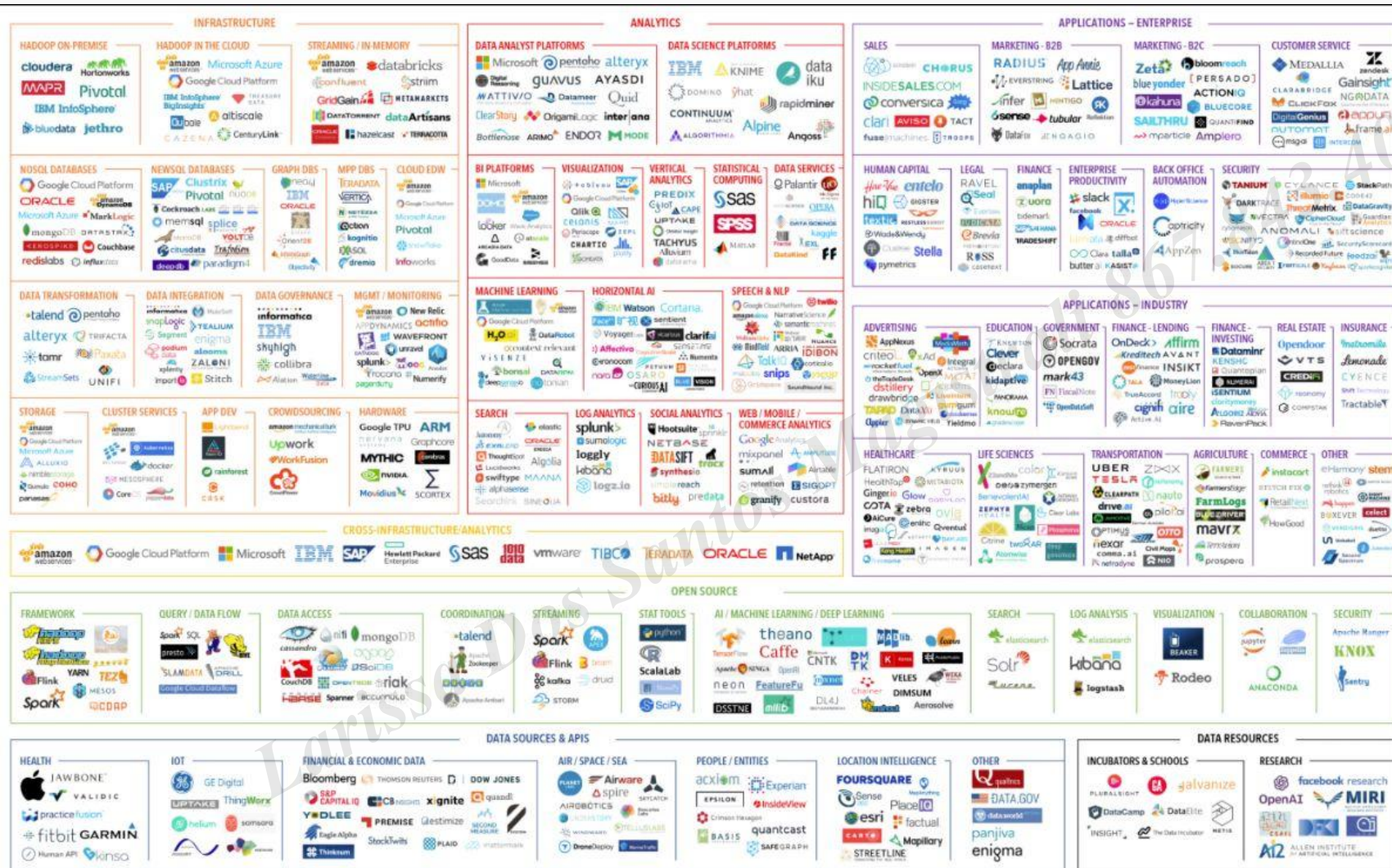
Empoderar

O que é: Compartilhar os resultados e insights com as partes interessadas relevantes de forma clara e compreensível.

Métodos: Criação de painéis, relatórios e apresentações.

Objetivo: Permitir que outras pessoas dentro da organização entendam a história dos dados e tomem decisões informadas, promovendo uma cultura orientada por dados.

Data Science Tech Stack



<https://www.business-science.io/business/2019/11/15/intimatedata-science-tech-stack.html>

Coletar

Processar

Armazenar

Analisar

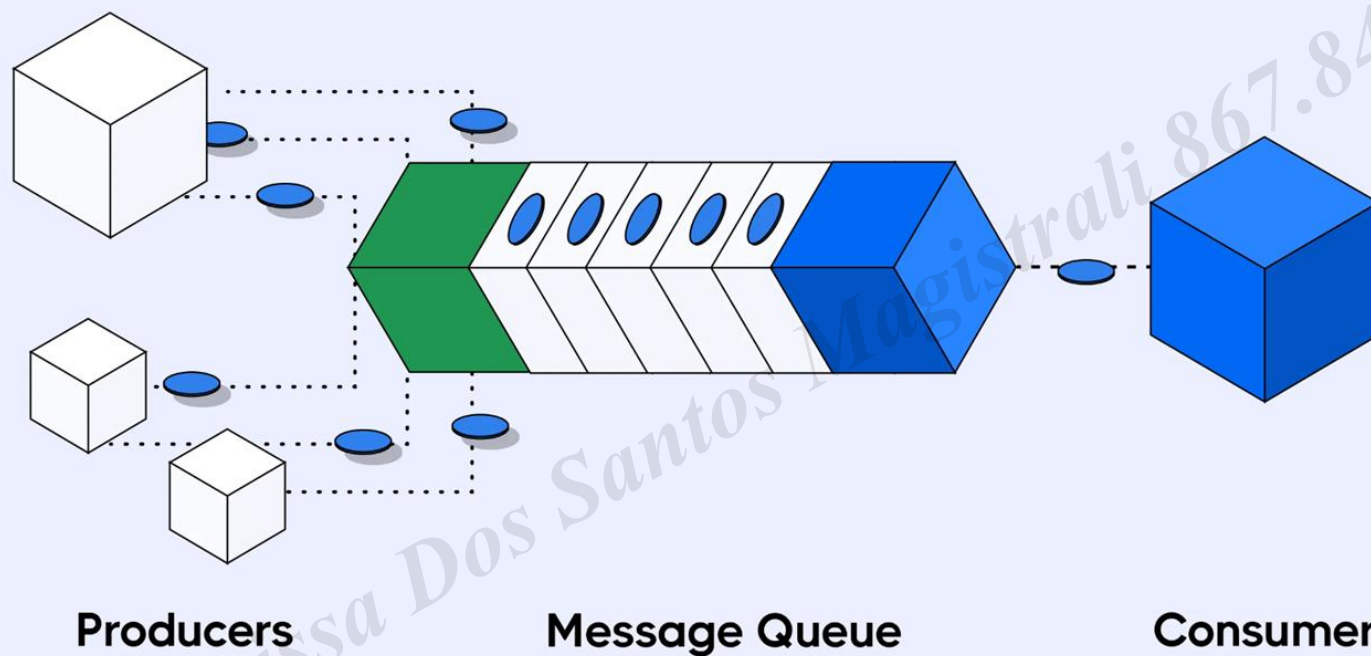
Ativar

Empoderar

Streaming de mensagens vs. fila de mensagens

Tipo de plataforma	Foco principal	Tratamento de dados	Características principais
Plataforma de streaming de eventos	Processamento em tempo real de fluxos de dados contínuos.	Processa eventos conforme eles ocorrem.	Alto rendimento, baixa latência, dados frequentemente imutáveis e reproduzíveis. Suporta um modelo de publicação-assinatura, no qual vários aplicativos podem consumir o mesmo fluxo de dados.
Fila de mensagens	Entrega confiável de mensagens entre aplicativos.	Normalmente remove mensagens quando elas são consumidas por um único destinatário.	Garante que uma tarefa seja executada uma única vez. Ideal para desacoplar tarefas e gerenciar cargas de trabalho. Menos focado na retenção de dados a longo prazo ou na análise complexa do próprio fluxo de dados.

Fila de mensagens



<https://hookdeck.com/blog/introduction-message-queue>

Benefícios de uma plataforma de fila de mensagens

Plataformas de Fila de Mensagens (MQ) tornaram-se uma ferramenta indispensável para a construção de aplicações robustas, escaláveis e eficientes. Ao permitir a **comunicação assíncrona** entre diferentes partes de um sistema, ou entre sistemas totalmente separados, as MQs oferecem uma alternativa poderosa à comunicação direta e síncrona, gerando uma série de benefícios significativos.

Maior confiabilidade do sistema e tolerância a falhas: Um dos benefícios mais importantes da utilização de uma fila de mensagens é a **melhoria drástica na confiabilidade do sistema**. Em um sistema fortemente acoplado, **onde os componentes se comunicam diretamente, a falha de um componente pode ter um efeito cascata**, derrubando toda a aplicação.

Escalabilidade e desempenho aprimorados: filas de mensagens são fundamentais na construção de aplicações altamente escaláveis. Elas permitem a dissociação de produtores e consumidores, que podem então ser escalonados de forma independente.

- **Balanceamento de Carga:** Uma única fila pode ter vários consumidores. A plataforma MQ pode distribuir as mensagens entre esses consumidores, equilibrando a carga de forma eficaz.
- **Lidando com Picos de Tráfego:** Para aplicações que sofrem com cargas flutuantes, como plataformas de e-commerce durante uma liquidação, **as MQs funcionam como um buffer**. Elas podem absorver picos repentinos de solicitações, enfileirando-as para processamento em um ritmo administrável para os sistemas de back-end, evitando sobrecarga e travamentos do sistema.

Captura de Dados Alterados (CDC) - Streaming

O CDC captura modificações de dados em tempo real e as envia aos sistemas posteriores. Pense nele como um sistema de notificação para seu banco de dados que informa exatamente o que mudou, quando mudou e como mudou.

Ele monitora e captura três tipos de operações de dados:

- INSERIR: Um novo registro é adicionado.
- ATUALIZAÇÃO: Um registro existente é modificado.
- DELETE: Um registro é removido.

Conceito central

Em vez de copiar periodicamente um conjunto de dados inteiro, o CDC envia apenas as alterações incrementais. Isso é muito mais eficiente e fornece um fluxo de informações em tempo real.

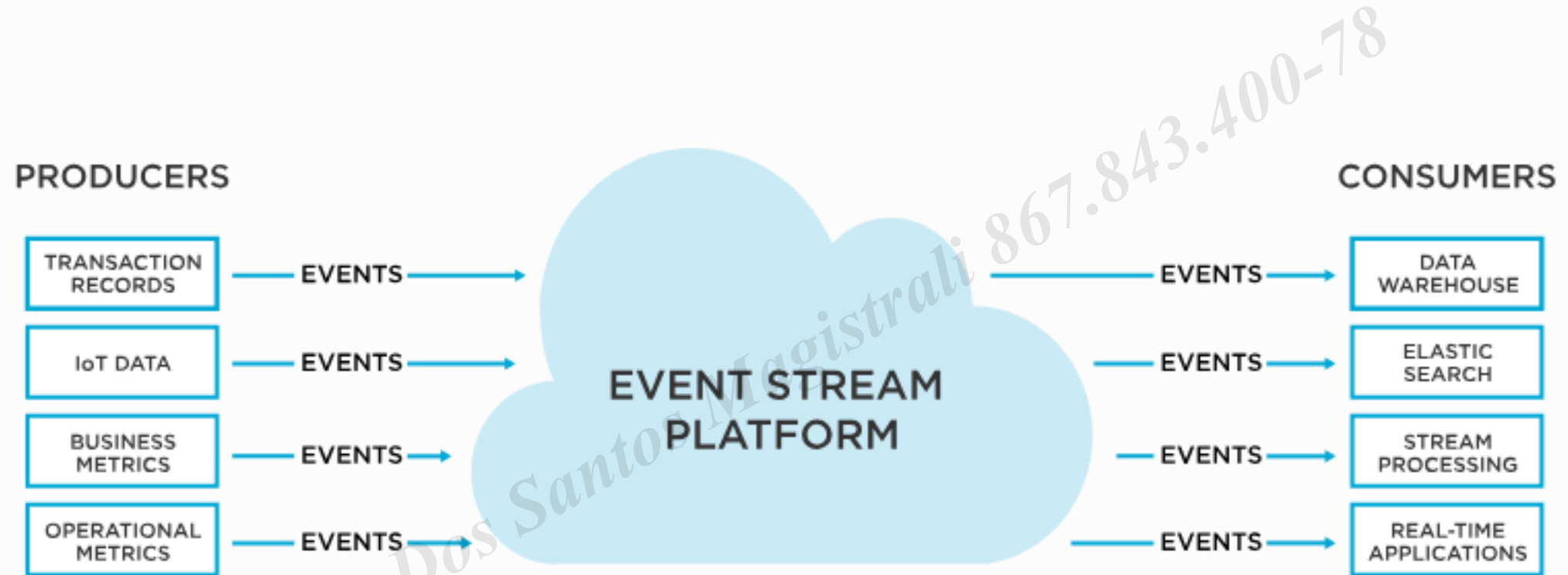
Banco de dados de origem -> Processo CDC -> Sistemas de destino (Data Warehouse, Analytics, Apps)

Como o CDC funciona e por que é útil

Métodos comuns do CDC

1. CDC baseado em log (mais popular):
 - a. Lê o log de transações nativo do banco de dados (por exemplo, WAL do PostgreSQL, Binlog do MySQL).
 - b. Prós: Impacto muito baixo no desempenho do banco de dados de origem; captura todas as alterações com precisão.
2. CDC baseado em gatilhos:
 - a. Usa gatilhos de banco de dados (procedimentos armazenados especiais) que são disparados automaticamente em eventos INSERT, UPDATE ou DELETE.
 - b. Contra: Pode adicionar uma sobrecarga de desempenho significativa ao banco de dados de origem.
3. CDC baseado em consulta:
 - a. Consulta periodicamente as tabelas de origem para obter o registro de data e hora da "última atualização" ou o número da versão.
 - b. Contra: Pode perder alterações intermediárias e não pode capturar DELETES sem lógica extra.

Plataforma de streaming de eventos



<https://www.tibco.com/glossary/what-is-event-streaming>

Benefícios de uma plataforma de streaming de eventos

Insights e ações em tempo real: você pode analisar dados e tomar decisões quase instantaneamente. Isso é crucial para aplicações como detecção de fraudes, marketing em tempo real e monitoramento de sistemas críticos.

Escalabilidade: As plataformas de streaming de eventos são projetadas para lidar com grandes volumes de dados de diversas fontes sem perda de velocidade. Elas podem ser escalonadas horizontalmente, o que significa que você pode adicionar mais recursos à medida que seus dados aumentam.

Desacoplamento de Sistemas: Diferentes aplicações podem publicar e assinar fluxos de eventos de forma independente. Esse acoplamento flexível torna o sistema geral mais resiliente e fácil de manter. Se um componente falhar, o sistema inteiro não será desativado.

Reprodutibilidade de dados: os eventos geralmente são armazenados por um período configurável. Isso permite que você "reproduza" dados históricos para testar novos aplicativos, recuperar-se de erros ou realizar novos tipos de análise em eventos passados.

Experiências aprimoradas do cliente: ao reagir ao comportamento do cliente em tempo real, você pode fornecer recomendações personalizadas, preços dinâmicos e suporte imediato.

Kafka

Larissa Dos Santos Magistrali 867.843.400-78

O que é Kafka?

- Kafka é uma plataforma de streaming distribuída que permite publicar, assinar, armazenar e processar fluxos de registros em tempo real.
- Ele foi originalmente desenvolvido pelo LinkedIn e agora é um projeto de código aberto mantido pela Apache Software Foundation.

Por que usar o Kafka?

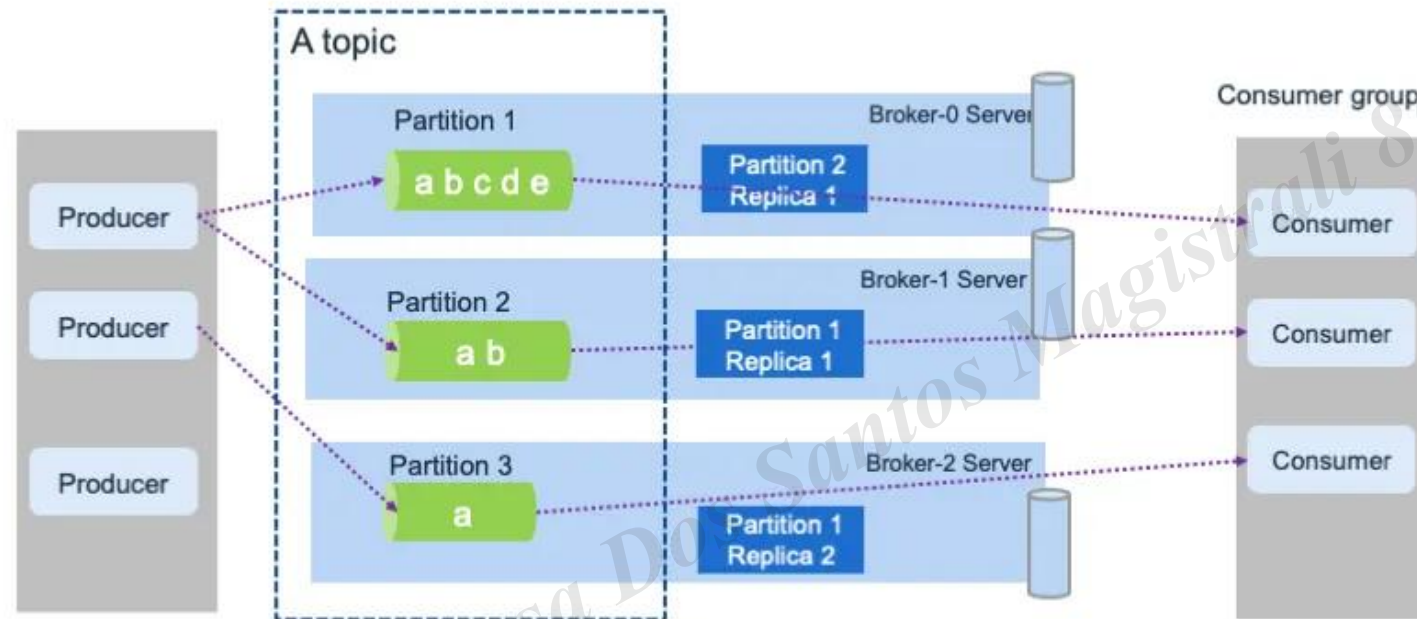
- Escalabilidade: O Kafka foi projetado para ser altamente escalável, capaz de lidar com trilhões de eventos por dia.
- Durabilidade: os dados são persistidos no disco, tornando-o um sistema de mensagens confiável e durável.
- Alto rendimento: o Kafka pode lidar com fluxos de dados de alta velocidade com baixa latência.
- Desacoplamento de sistemas: permite o desacoplamento de produtores e consumidores de dados, possibilitando uma arquitetura mais flexível e resiliente.

Casos de uso comuns:

- **Pipelines de dados em tempo real:** criação de pipelines de dados para mover dados entre aplicativos e sistemas.
- **Arquitetura orientada a eventos:** potencializando aplicativos que reagem a eventos conforme eles acontecem.
- **Agregação de logs:** coleta e processamento de logs de vários serviços.
- **Processamento de fluxo:** criação de aplicativos que consomem, processam e analisam fluxos de dados em tempo real.

Conceitos Básicos do Kafka

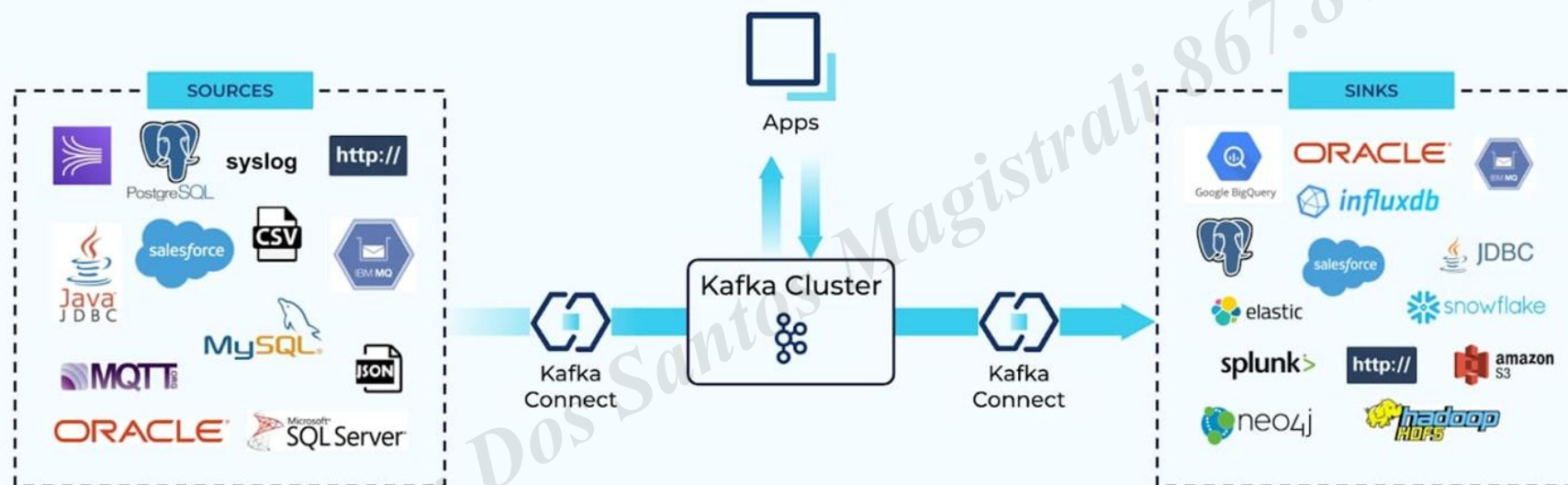
Kafka Architecture



Self-Managed Metadata Quorum

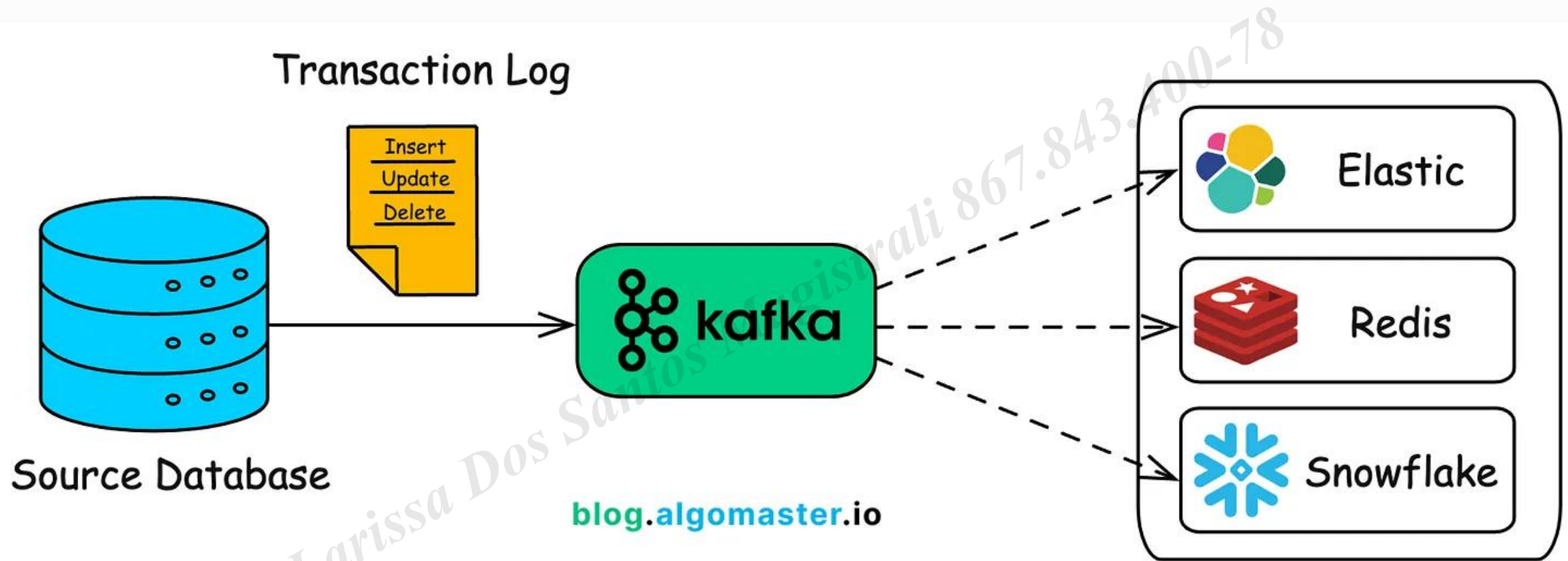
<https://medium.com/@cobch7/kafka-architecture-43333849e0f4>

Ingest Data from Upstream Systems



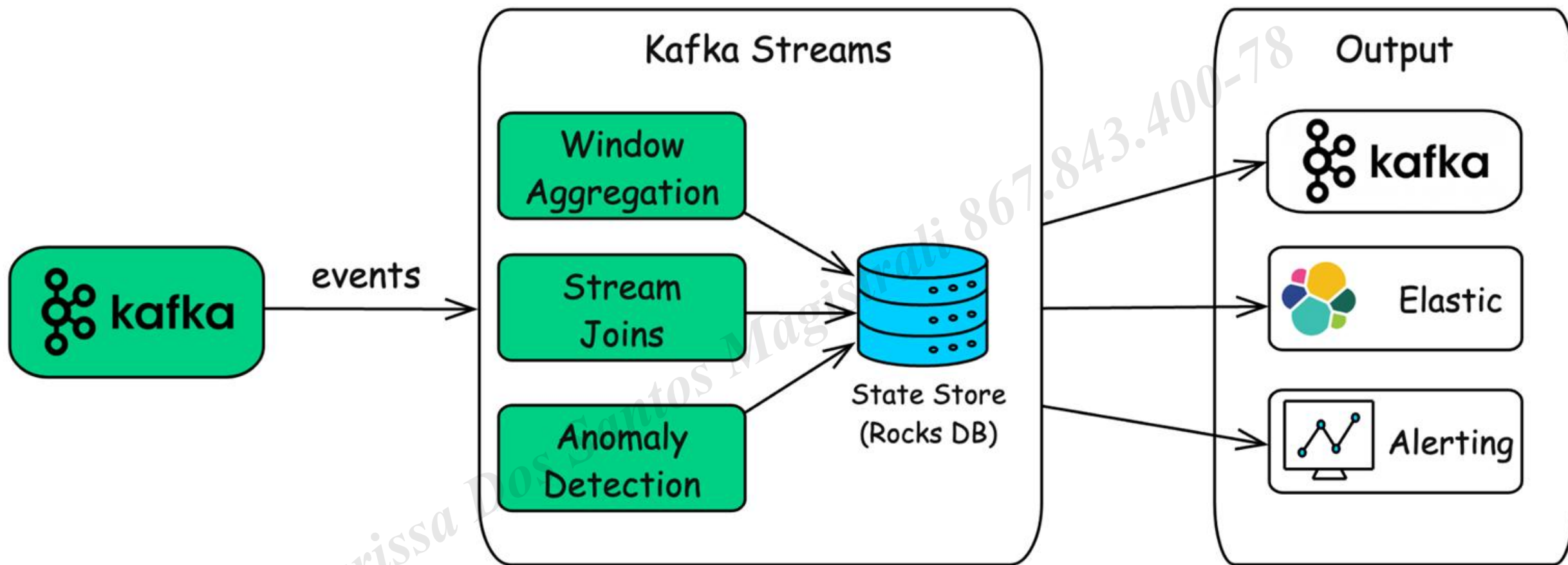
@TheDanicaFine | developer.confluent.io

<https://developer.confluent.io/courses/kafka-connect/intro/>



<https://blog.algomaster.io/p/top-10-kafka-use-cases>

Kafka Streams



blog.algomaster.io

<https://blog.algomaster.io/p/top-10-kafka-use-cases>

Coletar

Processar

Armazenar

Analisar

Ativar

Empoderar

Processamento em Lote (Batch) vs. Tempo Real (Real-Time)

Processamento em Lote (Batch)

O processamento em lote, ou batch processing, é um método no qual um grande volume de dados é coletado, armazenado e, posteriormente, processado em conjunto, de uma só vez. As tarefas são executadas em "lotes" ou grupos, geralmente em períodos de baixa atividade do sistema, como durante a noite, para otimizar o uso de recursos computacionais.

Características Principais:

- **Alto Volume de Dados:** Ideal para lidar com grandes conjuntos de dados que não requerem análise imediata.
- **Latência Elevada:** Existe um atraso significativo entre a coleta dos dados e a obtenção dos resultados, que pode variar de minutos a horas.
- **Processamento Agendado:** As tarefas são acionadas por agendamento (ex: a cada 24 horas) ou quando um determinado volume de dados é atingido.
- **Custo-Benefício:** Geralmente mais econômico, pois utiliza os recursos de computação de forma mais eficiente e em momentos de menor demanda.

Exemplos de Uso:

- Processamento de Folha de Pagamento: Cálculo dos salários dos funcionários realizado mensalmente.
- Sistemas de Faturamento: Geração de faturas e cobranças para clientes ao final de um ciclo.
- Relatórios de Vendas: Consolidação e análise dos dados de vendas do dia, semana ou mês anterior.
- Processos de ETL (Extração, Transformação e Carga): Movimentação de grandes volumes de dados de sistemas operacionais para um data warehouse.

Processamento em Tempo Real (Real-Time)

O processamento em tempo real, também conhecido como stream processing (processamento de fluxo), trata os dados de forma contínua, à medida que são gerados. A análise e a resposta aos eventos ocorrem de maneira instantânea ou com uma latência muito baixa, geralmente na ordem de segundos ou milissegundos.

Características Principais:

- **Baixo Volume de Dados (por vez):** Processa pequenos pacotes de dados ou eventos individuais de forma contínua.
- **Latência Mínima:** A resposta é quase imediata, permitindo a tomada de decisões instantâneas.
- **Processamento Contínuo:** O sistema está sempre ativo, processando os dados assim que eles chegam.
- **Custo Mais Elevado:** Exige maior capacidade computacional e sistemas mais complexos para garantir o processamento contínuo e a baixa latência.

Exemplos de Uso:

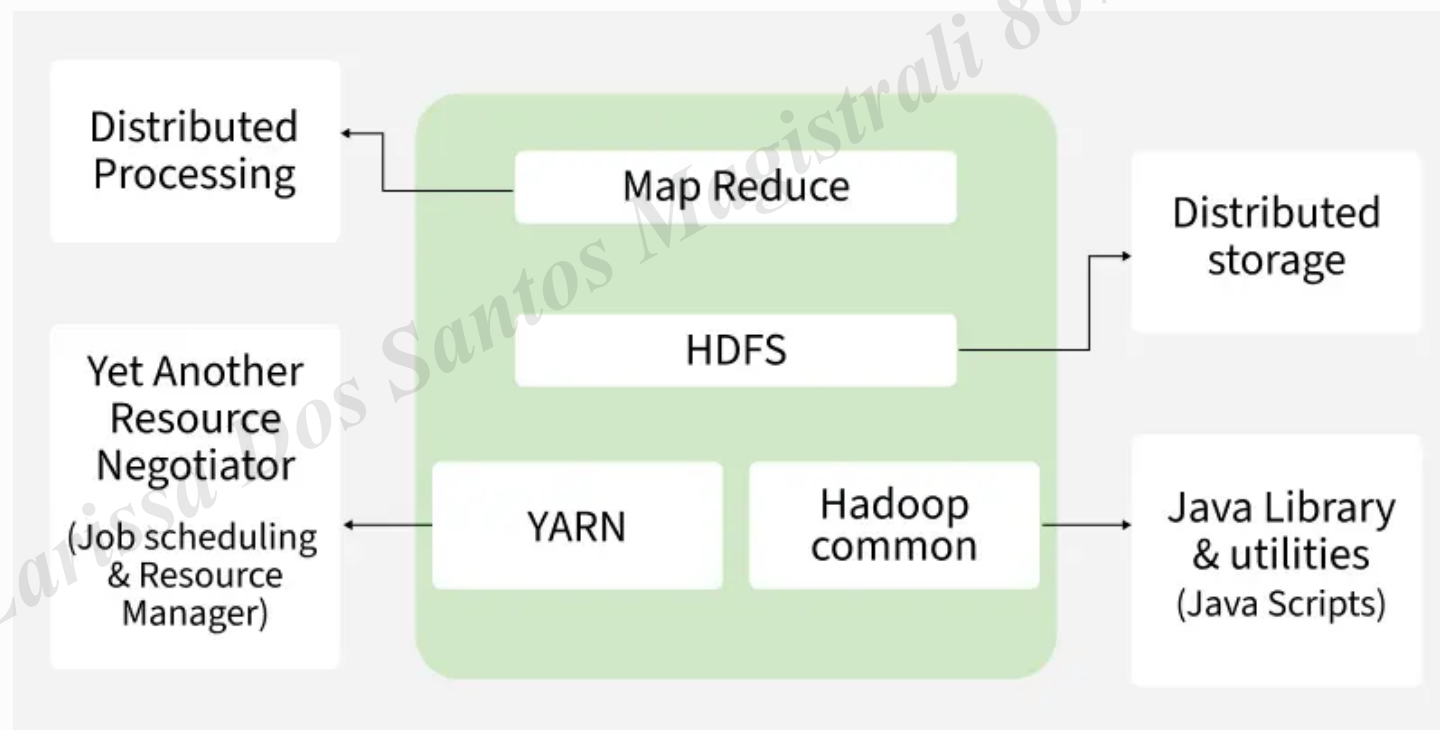
- **Detecção de Fraudes em Cartões de Crédito:** Análise de transações em tempo real para identificar e bloquear atividades suspeitas.
- **Monitoramento de Redes Sociais:** Análise de menções e sentimentos sobre uma marca para respostas rápidas.
- **Recomendações de Produtos em E-commerce:** Sugestão de itens para o cliente com base na sua navegação e comportamento em tempo real.
- **Monitoramento de Pacientes:** Análise contínua dos sinais vitais de um paciente em um hospital para alertar a equipe médica sobre qualquer anomalia.

Hadoop

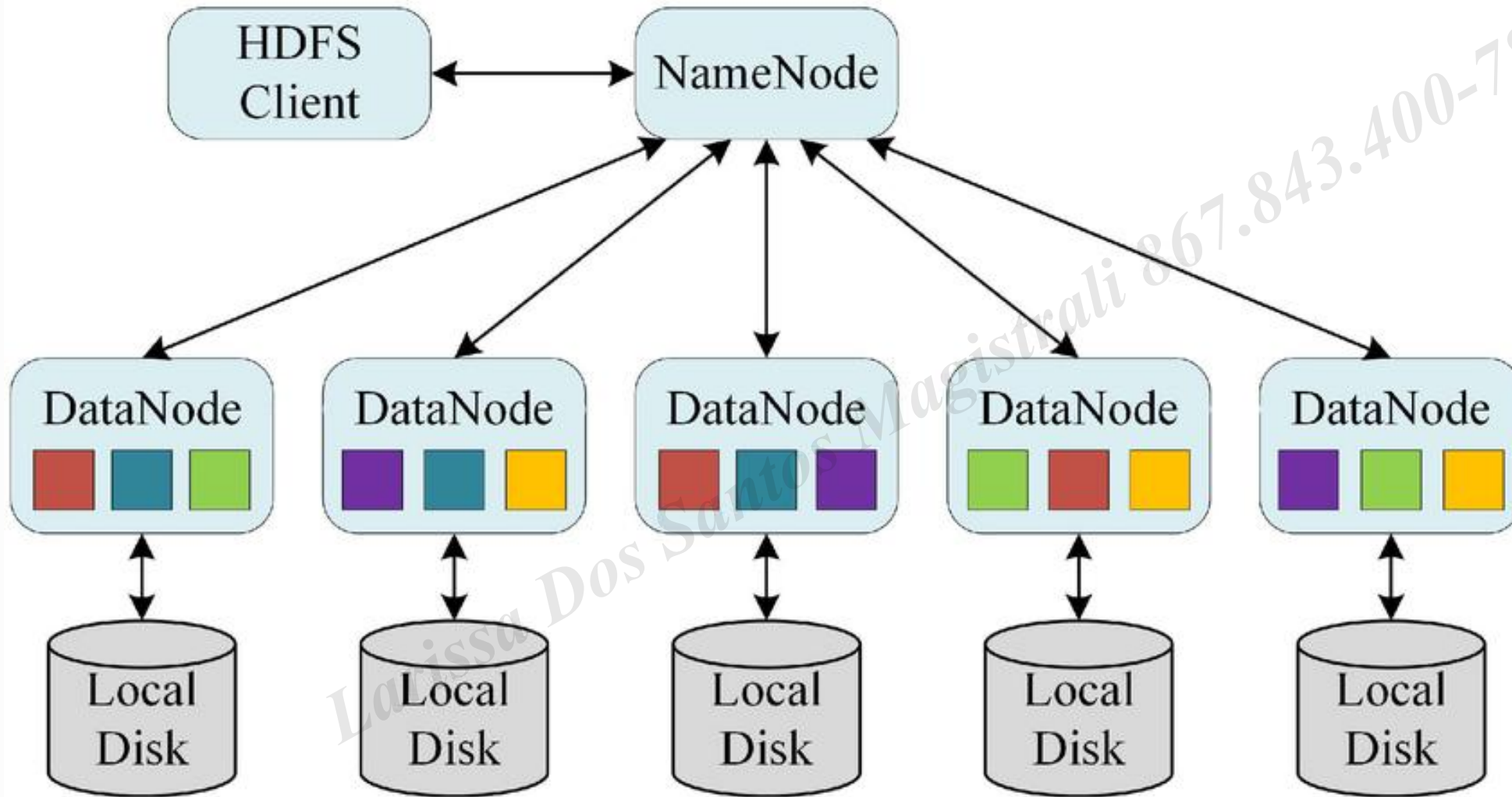
Larissa Dos Santos Magistrali 867.843.400-78

Introdução ao Hadoop

Hadoop é um **framework de código aberto** baseado em Java que gerencia o **armazenamento e o processamento de grandes volumes de dados** para aplicações. O **Hadoop utiliza armazenamento distribuído** e processamento paralelo para lidar com big data e tarefas analíticas, dividindo as cargas de trabalho em cargas menores que podem ser executadas simultaneamente.



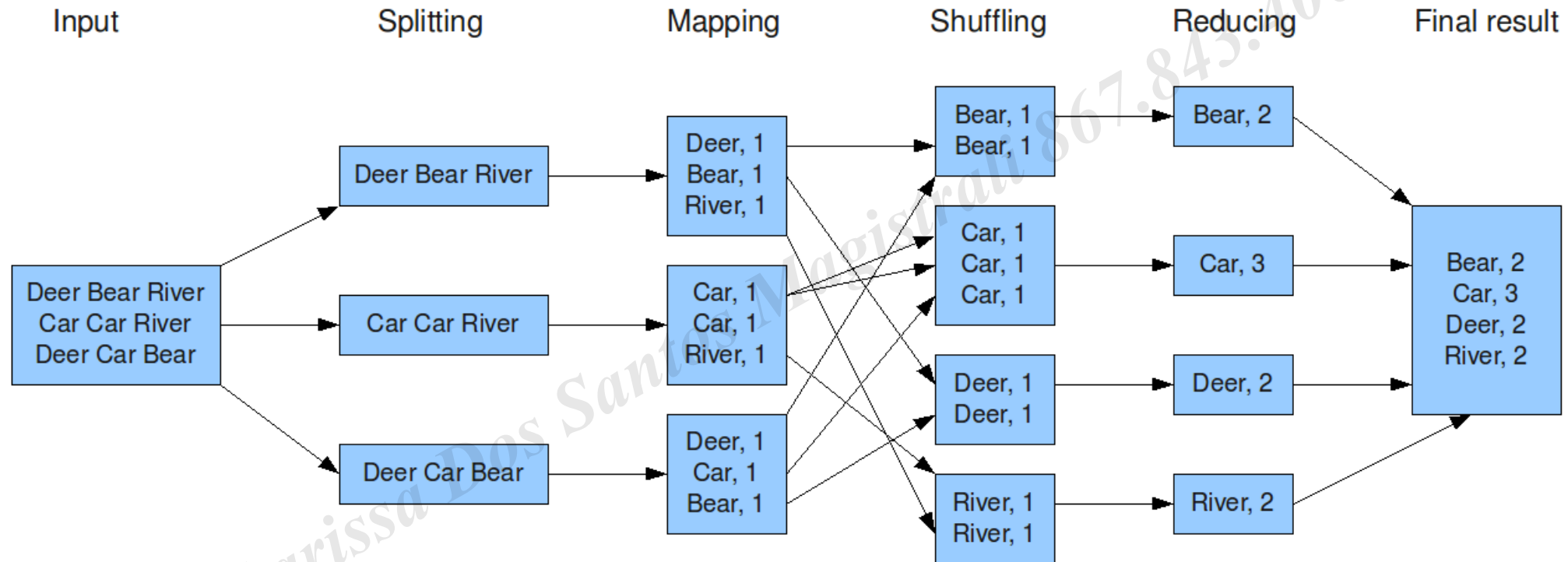
HDFS



https://www.researchgate.net/figure/The-overview-of-the-Hadoop-Distributed-File-System-HDFS_fig4_348387085

Example

The overall MapReduce word count process



<https://www.todaysoftmag.com/article/1358/hadoop-mapreduce-deep-diving-and-tuning>

Pseudo Code Example

```
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "word count");

job.setJarByClass(WordCount.class);

job.setMapperClass(WordCountMapper.class);
job.setCombinerClass(WordCountReducer.class);
job.setReducerClass(WordCountReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
```

```
public class WordCountReducer extends Reducer<Text,
IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();
    public void reduce(Text key, Iterable<IntWritable>
values, Context context) throws IOException,
InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

Apache Spark

Larissa Dos Santos Magistrali 867.843.400-78

What is Apache Spark

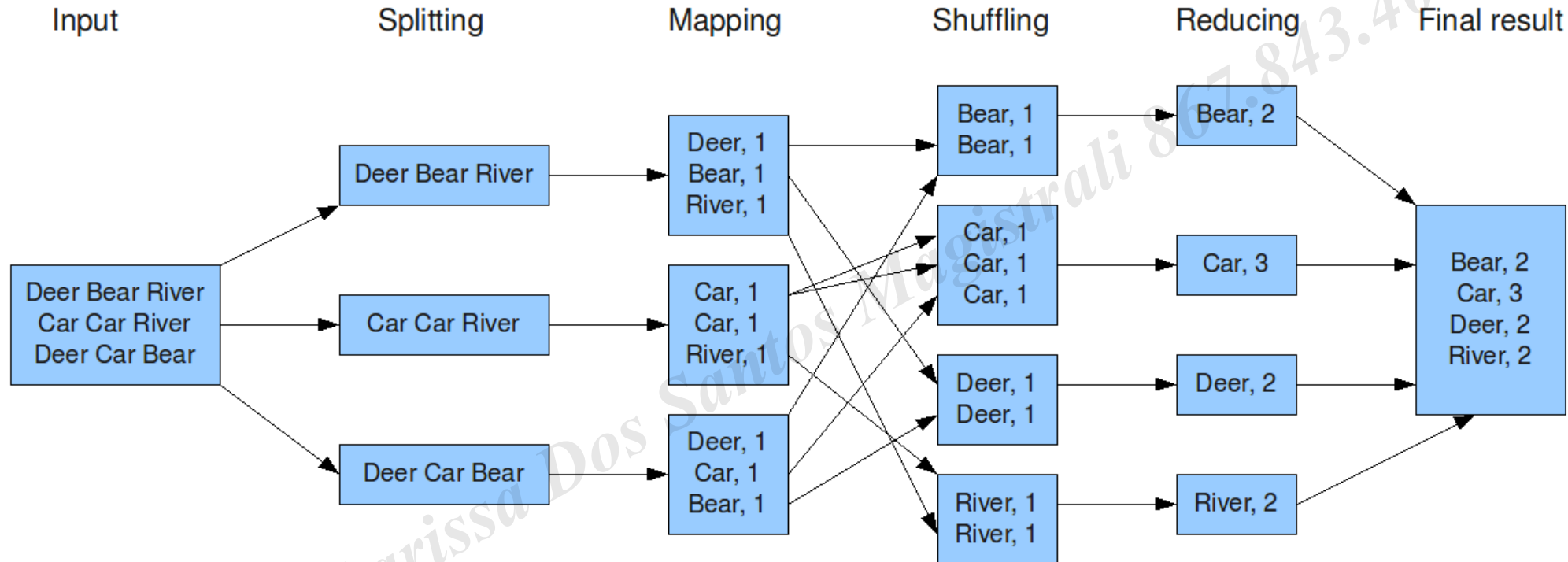
O que é Apache Spark

O Apache Spark é um poderoso mecanismo de análise unificado de código aberto, projetado para processamento de dados em larga escala. Ele fornece um modelo de programação simples e expressivo que suporta uma ampla gama de aplicações, incluindo ETL, consultas interativas, aprendizado de máquina e processamento de fluxo em tempo real.

O Apache Spark **realiza processamento na memória**. Ele carrega dados na memória (RAM) e os mantém lá para operações subsequentes. Isso reduz drasticamente o tempo gasto na leitura e gravação em disco, tornando o Spark significativamente mais rápido, até 100 vezes mais rápido para determinados aplicativos. O modelo de processamento do Spark é baseado em Conjuntos de Dados Distribuídos Resilientes (RDDs), que são coleções de objetos imutáveis e tolerantes a falhas que podem ser processados em paralelo.

Exemplo (você já viu este slide antes?)

The overall MapReduce word count process



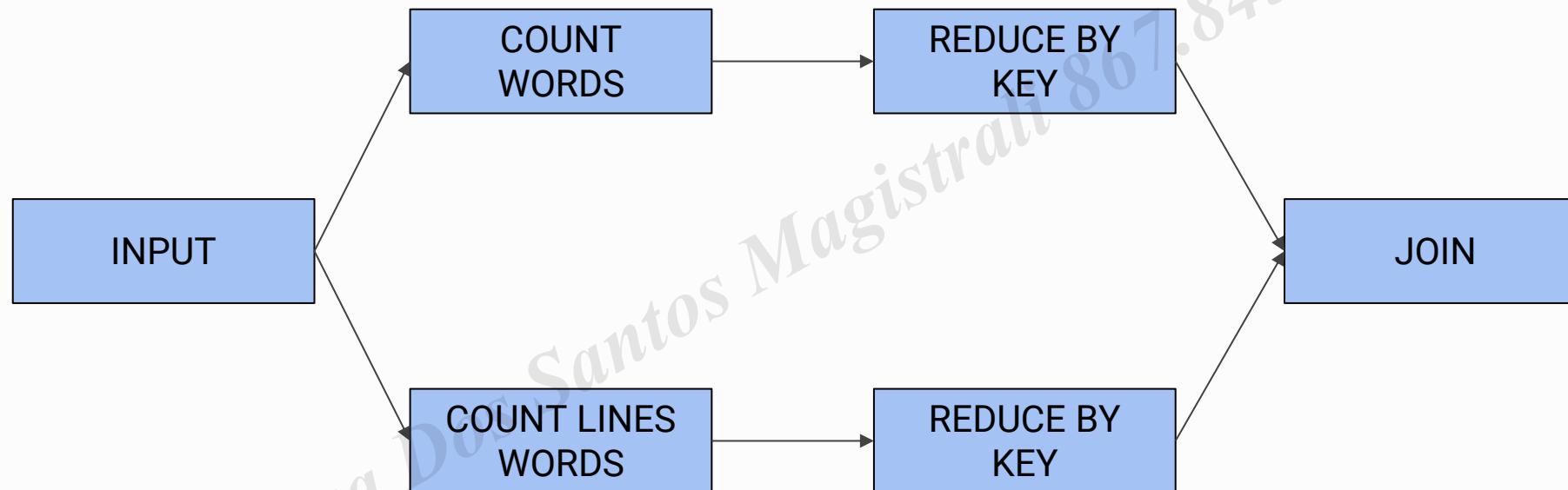
<https://www.todaysoftmag.com/article/1358/hadoop-mapreduce-deep-diving-and-tuning>

Pseudo Code

```
sc = SparkContext(conf=conf)
lines = sc.textFile(file_path)
words = lines.flatMap(lambda line: line.split())
word_counts = words.map(lambda word: (word.lower(), 1))
counts = word_counts.reduceByKey(lambda a, b: a + b)
output = counts.collect()
for (word, count) in output:
    print(f"{word}: {count}")
```

Larissa Dos Santos Magistrali 867.843.400-78

Apache Spark é um gráfico direcionado e acíclico de operações paralelas



Pseudo Code

```
sc = SparkContext(conf=conf)
lines_rdd = sc.textFile(file_path).cache()

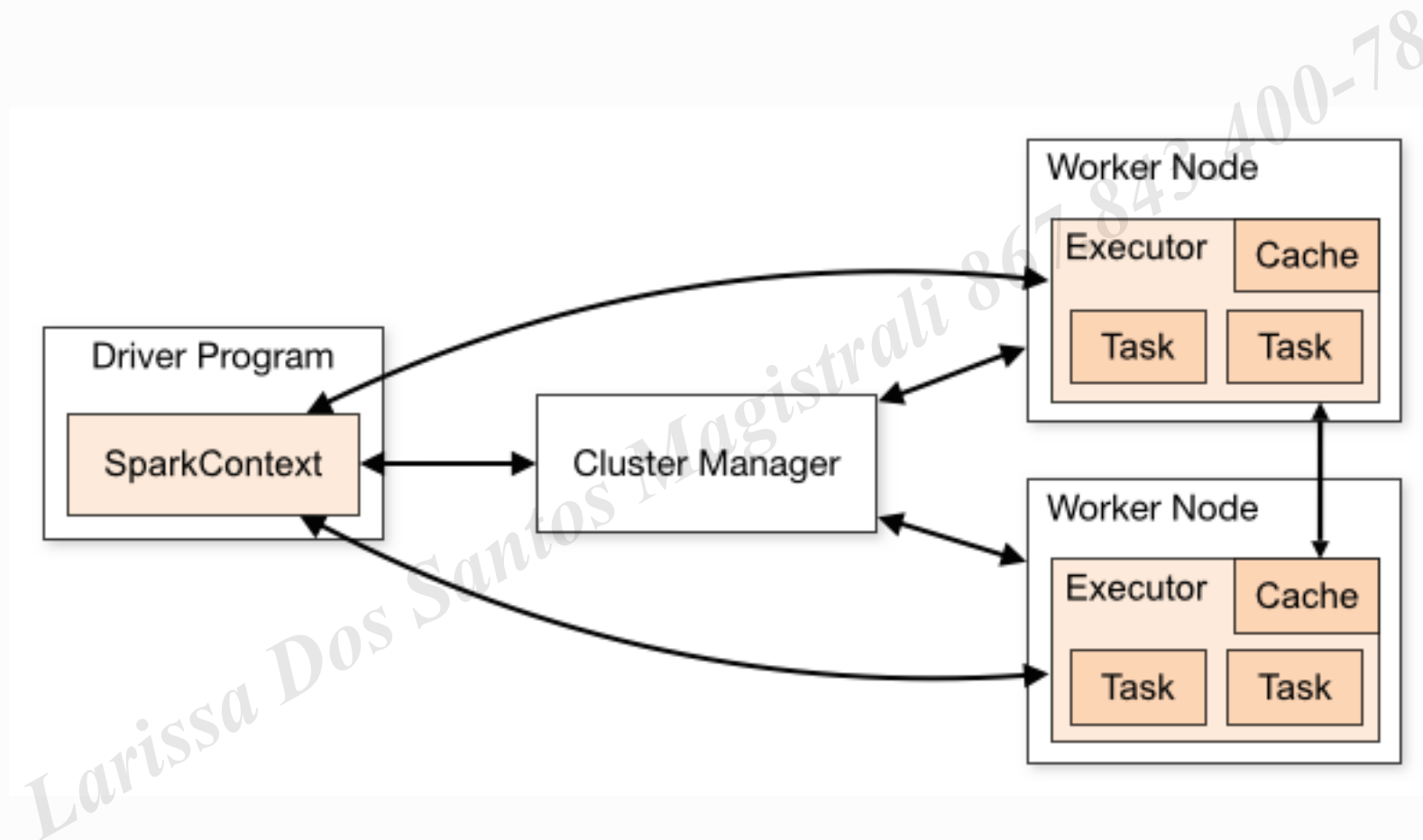
words = lines_rdd.flatMap(lambda line: [word for word in line.lower().split() if word])
total_counts = words.map(lambda word: (word, 1)).reduceByKey(lambda a, b: a + b)

lines_with_index = lines_rdd.zipWithIndex()
word_and_line_sets = lines_with_index.flatMap(
    lambda tpl: [(word, {tpl[1]}) for word in set(w for w in tpl[0].lower().split() if w)]
)
line_sets_per_word = word_and_line_sets.reduceByKey(lambda set1, set2: set1.union(set2))
line_counts = line_sets_per_word.mapValues(len)

joined_counts = total_counts.join(line_counts)
output = joined_counts.collect()

print("--- Word Analysis Results ---")
print(f"{'Word':<20} {'Total Count':<15} {'Line Count'}")
print("-" * 50)
```

Spark Architecture

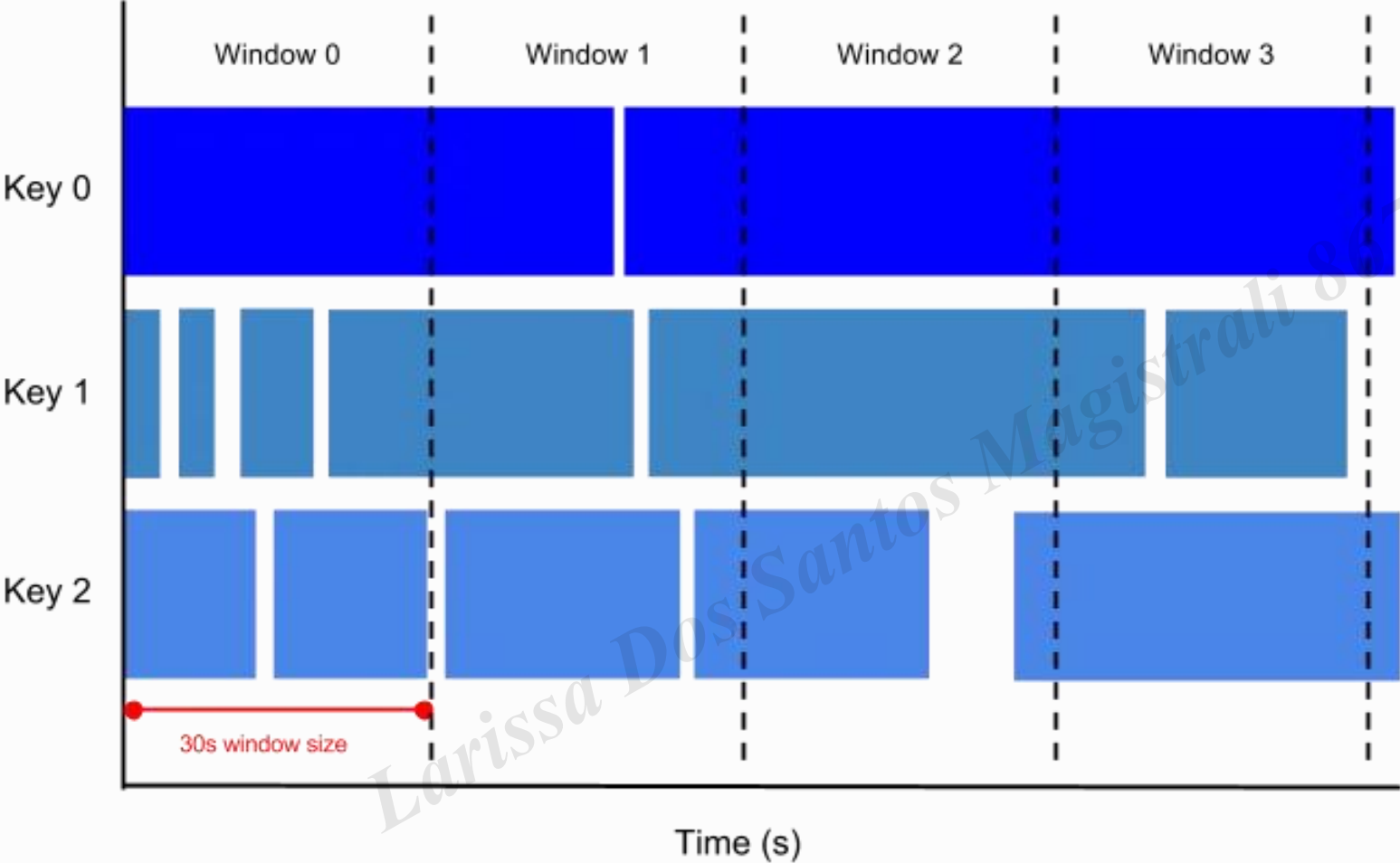


O que é Windowing

O windowing é um mecanismo que agrupa fluxos de dados ilimitados (dados contínuos) em blocos finitos, facilitando a análise e o processamento de dados. Ao definir janelas de tempo específicas, você pode dividir o fluxo contínuo em intervalos gerenciáveis, simplificando a análise e o processamento eficiente de dados.

Larissa Dos Santos Magistral 8621.943.400-78

Fixed Windows (Tumbling Windows)

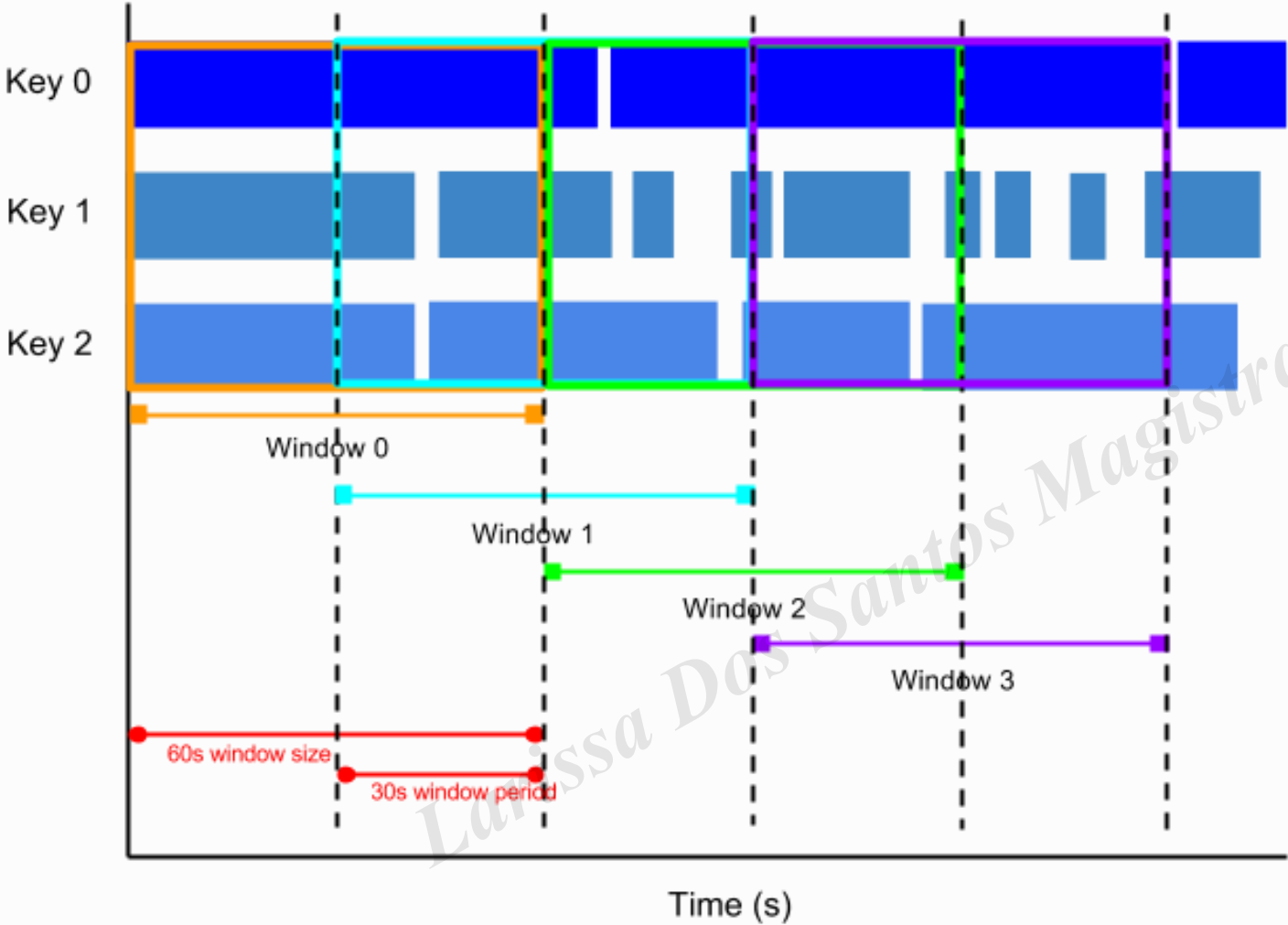


Time	Hashtag
13h:21m	#hadoop
13h:23m	#spark
13h:25m	#hadoop
13h:28m	#hadoop

5min window size

Hashtag	Count
#hadoop	2
#spark	1
#hadoop	1

Hopping Windows

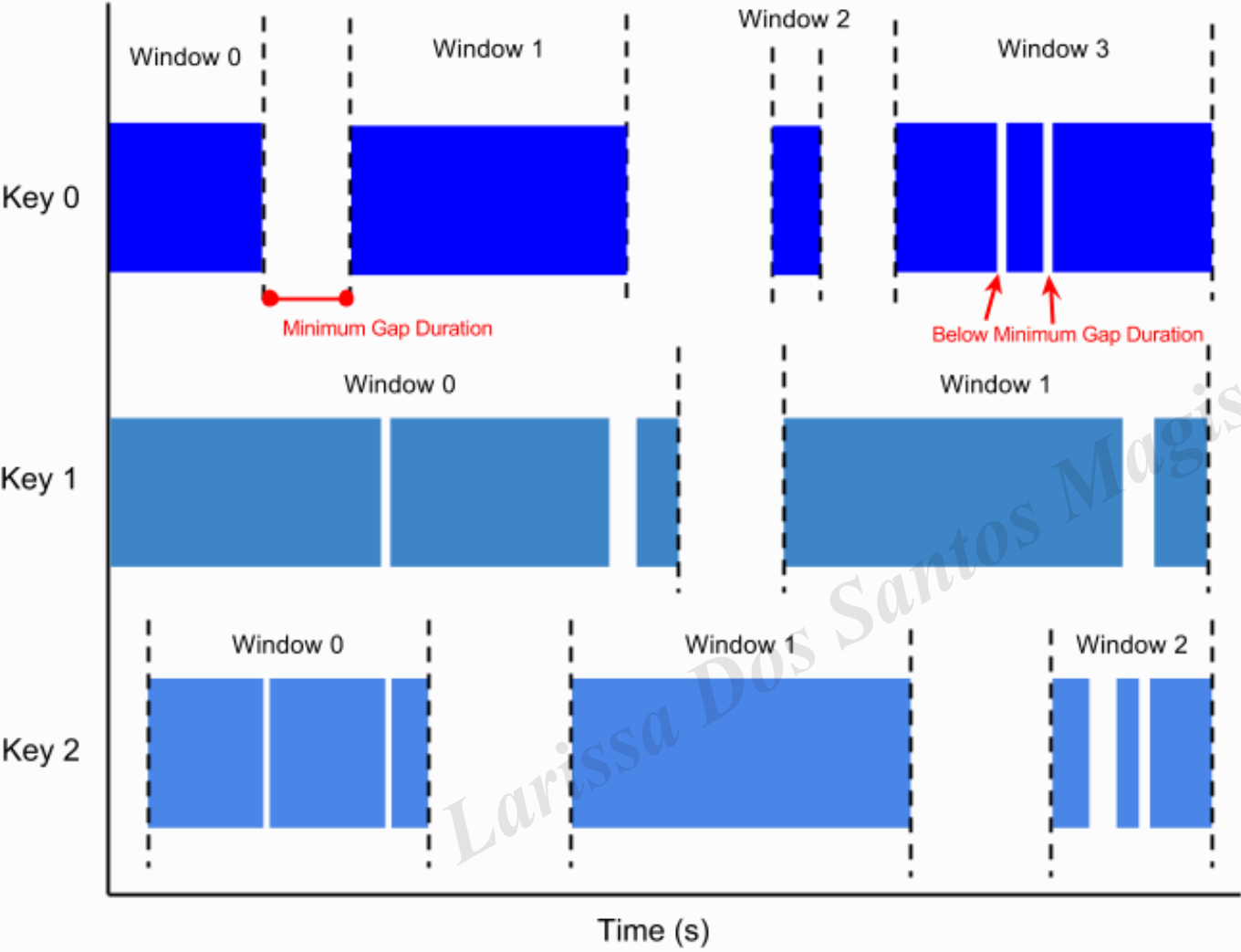


Time	Hashtag
13h:21m	#hadoop
13h:23m	#spark
13h:25m	#hadoop
13h:28m	#hadoop

5min window size
1min window period

Hashtag	Count
#hadoop	2
#spark	1
#hadoop	2

Session Windows



Time	Hashtag
13h:21m	#hadoop
13h:23m	#spark
13h:25m	#hadoop
13h:28m	#hadoop

3min window size

Hashtag	Count
#hadoop	1
#spark	1
#hadoop	2

Dados atrasados e marca d'água

Em qualquer sistema de processamento de dados, há um certo atraso entre o momento em que um evento de dados ocorre (o "tempo do evento", determinado pelo registro de data e hora no próprio elemento de dados) e o momento em que o elemento de dados é processado em qualquer estágio do pipeline (o "tempo de processamento", determinado pelo relógio do sistema que processa o elemento). Além disso, não há garantias de que os eventos de dados aparecerão no pipeline na mesma ordem em que foram gerados.

Time	Hashtag
13h:21m	#hadoop
13h:23m	#spark
13h:25m	#hadoop
13h:28m	#hadoop
13h:20m	#hadoop

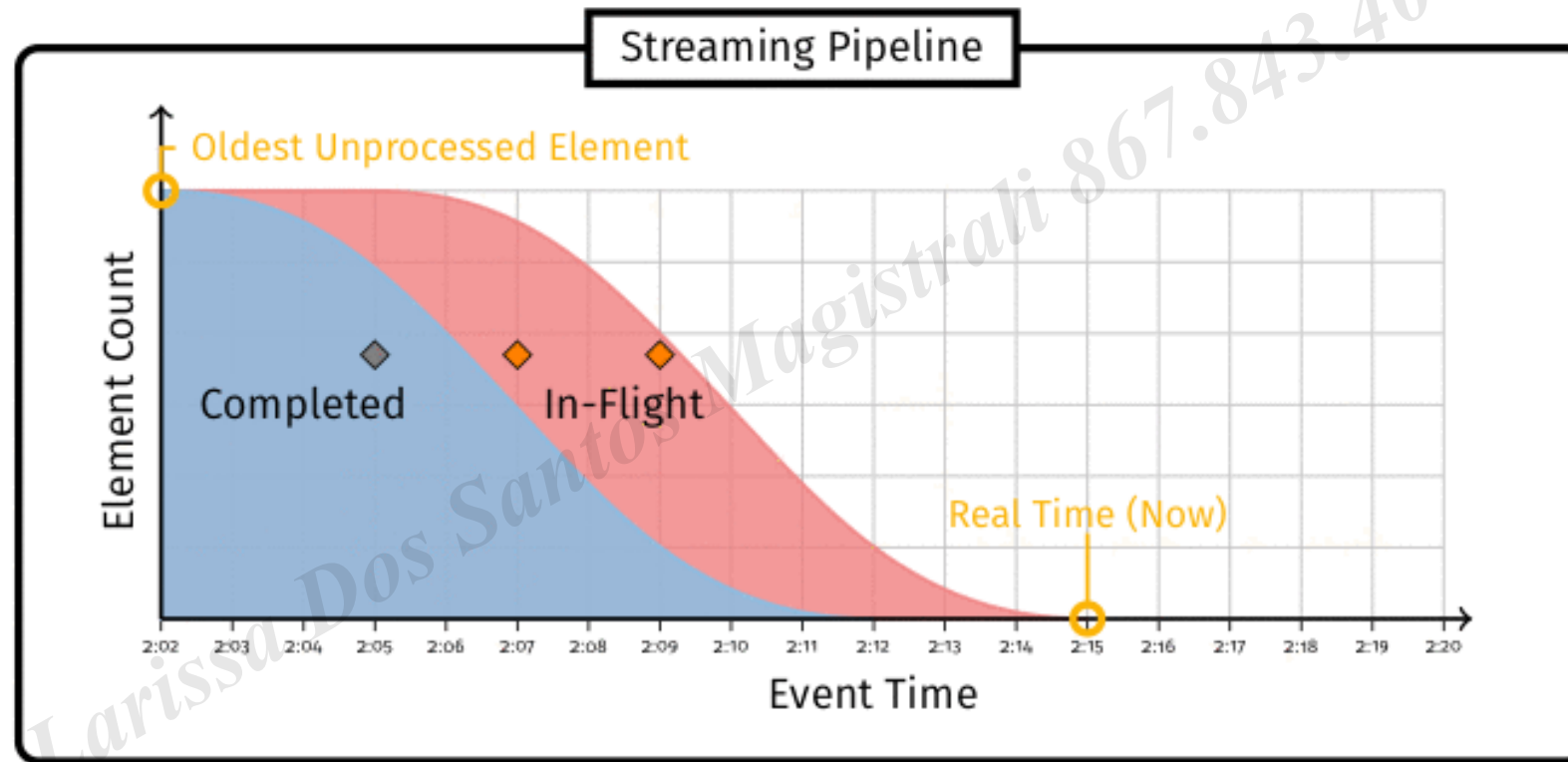
5min fixed window size
late threshold 0min

Hashtag	Count
#hadoop	2
#spark	1
#hadoop	1

5min fixed window size
late threshold 10min

Hashtag	Count
#hadoop	3
#spark	1
#hadoop	1

Dados atrasados e marca d'água



Coletar

Processar

Armazenar

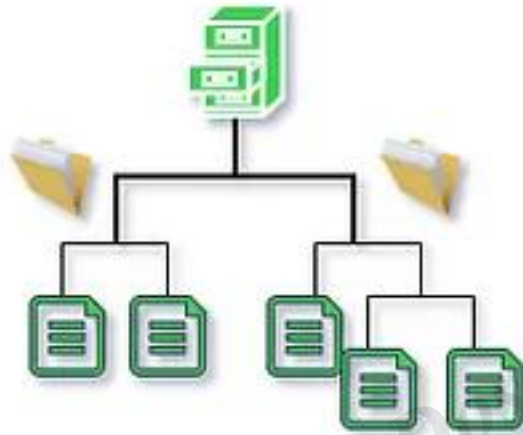
Analisar

Ativar

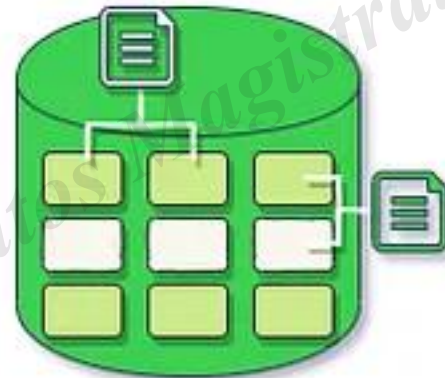
Empoderar

Tipos de armazenamento

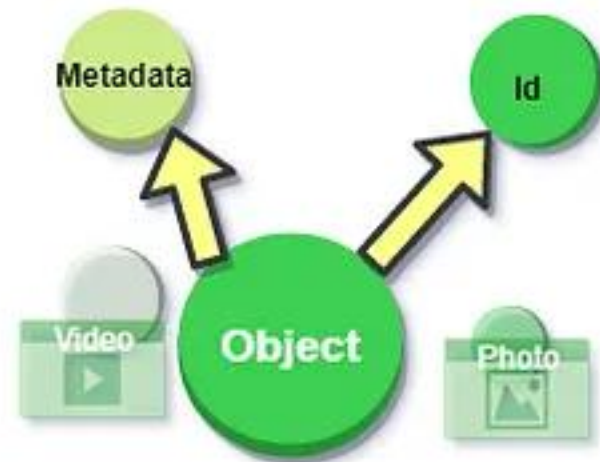
File Store



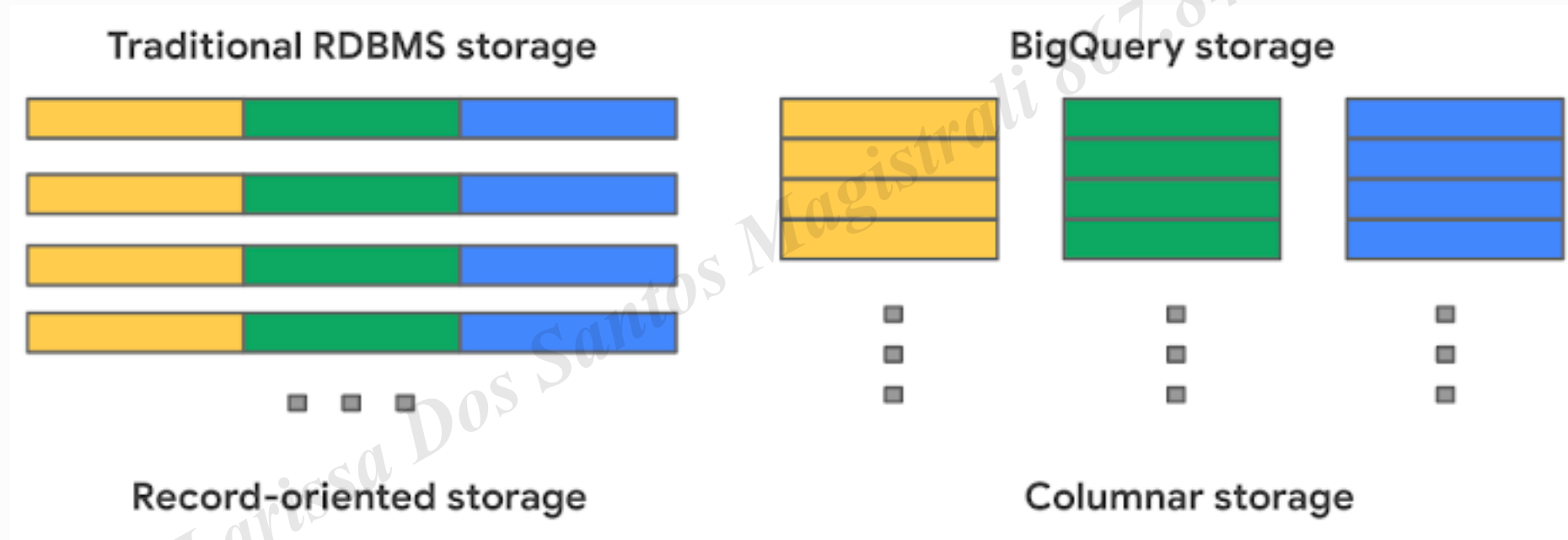
Block Store



Object Store



Armazenamento de tipo de arquivo



Armazenamento Colunar

UID	First Name	Last Name	Age	City	Purchases
349872349873	Ashuk	Patel	34	Chicago	[{ "pur_id": 3459872980, "UID": 349872349873, "Product": "Google Home", "Price": 100.01 }, { "pur_id": 38479782, "UID": 349872349873, "Product": "Pixel 4", "Price": 550.5 }, { "pur_id": 8937492, "UID": 349872349873, "Product": "Pixel 4 case", "Price": 20.23 }]
42398714298	Jessie	Walters	20	New York	[{ "pur_id": 3459872980, "UID": 349872349873, "Product": "Google Home", "Price": 100.01 }]
3498734871	Lisa	LaBlenc	54	Austin	[...]
34598792358	Greg	Smith	28	Boston	{...}

Apache Parquet

O Apache Parquet é um formato de arquivo de código aberto que armazena dados em formato colunar (em oposição ao formato de linha). Como formato de armazenamento de dados em coluna, ele oferece diversas vantagens em relação aos formatos baseados em linha para cargas de trabalho analíticas.

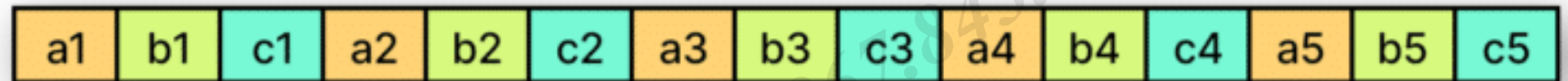
Larissa Dos Santos Magistrali 867.843.109-78

Apache Parquet

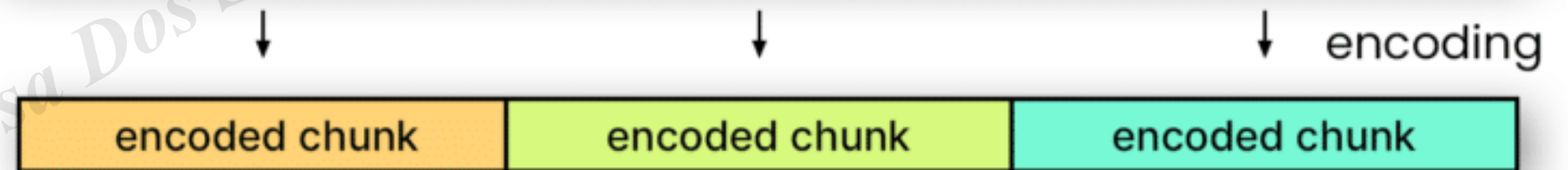
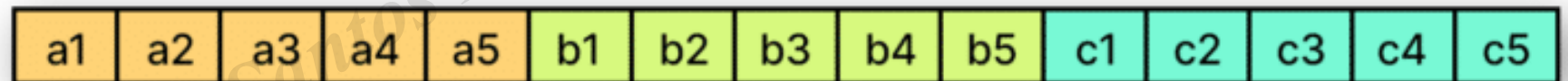
Logical table representation

a	b	c
a1	b1	c1
a2	b2	c2
a3	b3	c3
a4	b4	c4
a5	b5	c5

Row Layout



Column Layout



Benefícios do Apache Parquet

Alto desempenho

No Apache Parquet, os valores de **cada coluna são armazenados juntas no disco**. Como as consultas analíticas geralmente precisam apenas de um subconjunto de colunas para uma operação, isso reduz a quantidade de dados necessários para leitura.

Os arquivos Parquet também **contêm estatísticas sobre os dados armazenados**, como valores mínimos e máximos para os dados da coluna no arquivo e quantas linhas existem naquele segmento. Isso permite que os mecanismos ignorem segmentos inteiros ou arquivos inteiros com base na parte do conjunto de dados que o trabalho está procurando.

Esses dois recursos resultam em melhorias significativas de desempenho e redução de custos gerais, devido à redução na quantidade de dados que precisam ser lidos.

Compressão Eficiente

O Apache Parquet suporta compactação altamente eficiente. Muitos codecs de compactação são mais eficazes ao compactar dados semelhantes. O formato colunar do Parquet permite que colunas de dados semelhantes sejam compactadas juntas, resultando em maior eficiência.

Dados compactados são mais econômicos para armazenar do que dados brutos, portanto, usar o Parquet pode reduzir o custo de armazenamento de grandes conjuntos de dados. Dados compactados também apresentam melhor desempenho de leitura do que dados não compactados quando a E/S é o gargalo, o que pode ocorrer com frequência em cargas de trabalho analíticas. Portanto, dados compactados em arquivos Parquet também podem melhorar o desempenho.

Padrão da indústria

O Apache Parquet é o formato de arquivo colunar padrão da indústria. Arquivos Parquet podem ser lidos por praticamente todos os mecanismos e ferramentas disponíveis. Isso lhe proporciona interoperabilidade para usar diversas ferramentas hoje mesmo e a tranquilidade de poder aproveitar novos mecanismos e ferramentas que serão lançados amanhã, já que provavelmente serão compatíveis com Parquet.

Desvantagens do Apache Parquet

Arquivos baseados em binários não podem ser lidos por humanos

Parquet é um formato de arquivo baseado em binário (em vez de texto) otimizado para computadores, portanto, arquivos Parquet não são legíveis diretamente por humanos. Você não pode abrir um arquivo Parquet em um editor de texto como faria com um arquivo CSV e ver o que ele contém. Existem utilitários que convertem a representação binária para uma baseada em texto, como o parquet-tools, mas isso é uma etapa extra.

Tempos de gravação mais lentos

Arquivos Parquet podem ser mais lentos para gravar do que formatos de arquivo baseados em linhas, principalmente porque contêm metadados sobre o conteúdo do arquivo. Para fins analíticos, esses tempos de gravação mais lentos são amplamente compensados por tempos de leitura mais rápidos. No entanto, para situações em que a atualização dos dados e a latência dos eventos são mais importantes (por exemplo, na faixa de dezenas de milissegundos), pode valer a pena utilizar um formato baseado em linhas sem estatísticas, como Avro ou CSV.

Larissa Dos Santos MBAUSP
867.843.100-78

Apache Parquet vs. Apache Avro

O Apache **Avro também é um formato de arquivo binário**, assim como o Parquet. No entanto, o Avro é um formato de **arquivo baseado em linhas**, semelhante ao CSV, e foi projetado para **minimizar a latência de gravação**. Os arquivos Avro têm muito menos linhas por arquivo do que o Parquet, às vezes até apenas uma linha por arquivo. O caso de uso mais comum do Avro é o streaming de dados.

O Parquet tem as mesmas vantagens que o Avro e o CSV: é melhor para análises, tem melhor desempenho e é mais econômico.

Larissa Dos Santos Magistrali 867.843.400-78

Apache Iceberg

Larissa Dos Santos Magistrali 867.843.400-78

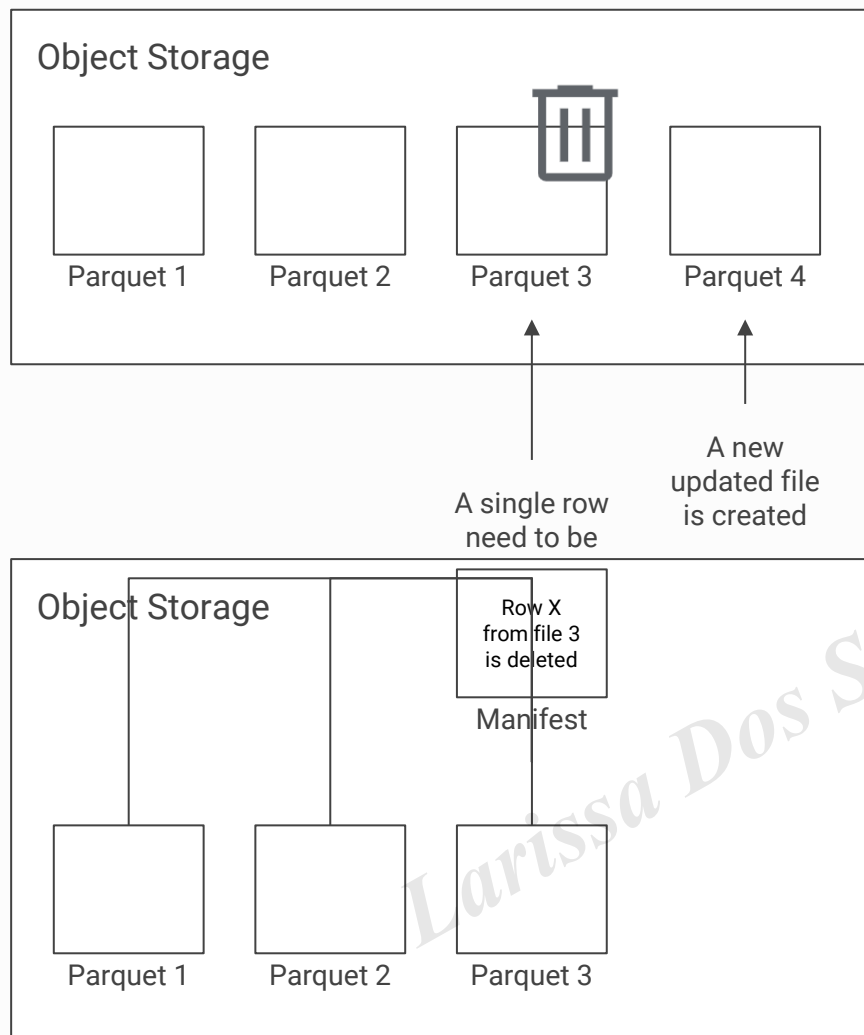
Apache Iceberg

O Apache Iceberg é um formato de tabela de código aberto projetado para conjuntos de dados analíticos de grande escala armazenados em data lakes. Ele atua como uma camada de metadados sobre arquivos de dados (como Parquet ou ORC) em um data lake, fornecendo recursos semelhantes aos de um banco de dados para conjuntos de dados massivos.

Os principais recursos e benefícios do Apache Iceberg incluem:

- **Transações ACID:** O Iceberg garante atomicidade, consistência, isolamento e durabilidade para operações de dados, trazendo garantias transacionais semelhantes às de banco de dados para data lakes.
- **Evolução do esquema:** Ele permite alterações flexíveis no esquema (adicionar, remover, renomear, reordenar colunas) sem exigir que os arquivos de dados subjacentes sejam reescritos.
- **Particionamento Oculto:** O Iceberg gerencia o particionamento de dados automaticamente, abstraindo o layout físico dos usuários e simplificando a otimização de consultas.
- **Viagem no tempo e retrocesso:** Ele mantém instantâneos do estado da tabela ao longo do tempo, permitindo que os usuários consultem dados históricos ou voltem para versões anteriores, se necessário.
- **Otimização de desempenho:** O gerenciamento detalhado de metadados e as estatísticas em nível de arquivo do Iceberg permitem planejamento e execução de consultas mais eficientes, melhorando o desempenho de cargas de trabalho analíticas.
- **Interoperabilidade:** Ele foi projetado para ser compatível com vários mecanismos e plataformas de computação, facilitando uma arquitetura "lakehouse" que combina os benefícios de data warehouses e data lakes.

O que é um formato de tabela?



Este exemplo simples expõe muitos problemas:

- **Eficiência:** É realmente ineficiente ficar apagando e recriando arquivos de parquet o tempo todo
- **Consistência:** após criar o Arquivo 4 e antes de excluir o Arquivo 3, uma carga de trabalho pode acessar os dados

Uma solução melhor é criar um arquivo manifesto com informações de metadados sobre seus arquivos parquet.

A combinação de arquivos parquet + arquivos manifest nos permite chamá-lo de "Formato de Tabela".

Como você pode imaginar, o Iceberg é muito mais complexo que este exemplo, ele permite que você tenha transações, atualizações, exclusões, partições, ACID, etc.

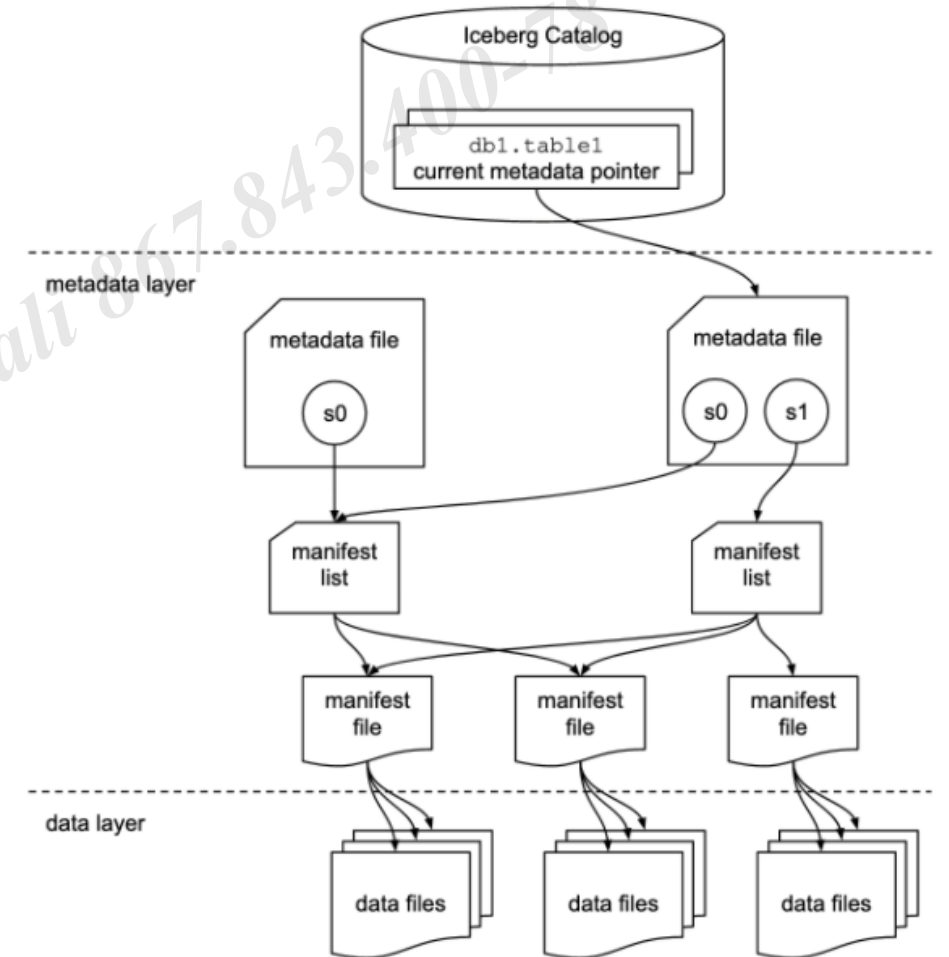
Apache Iceberg - Architecture

O iceberg é composto por três camadas

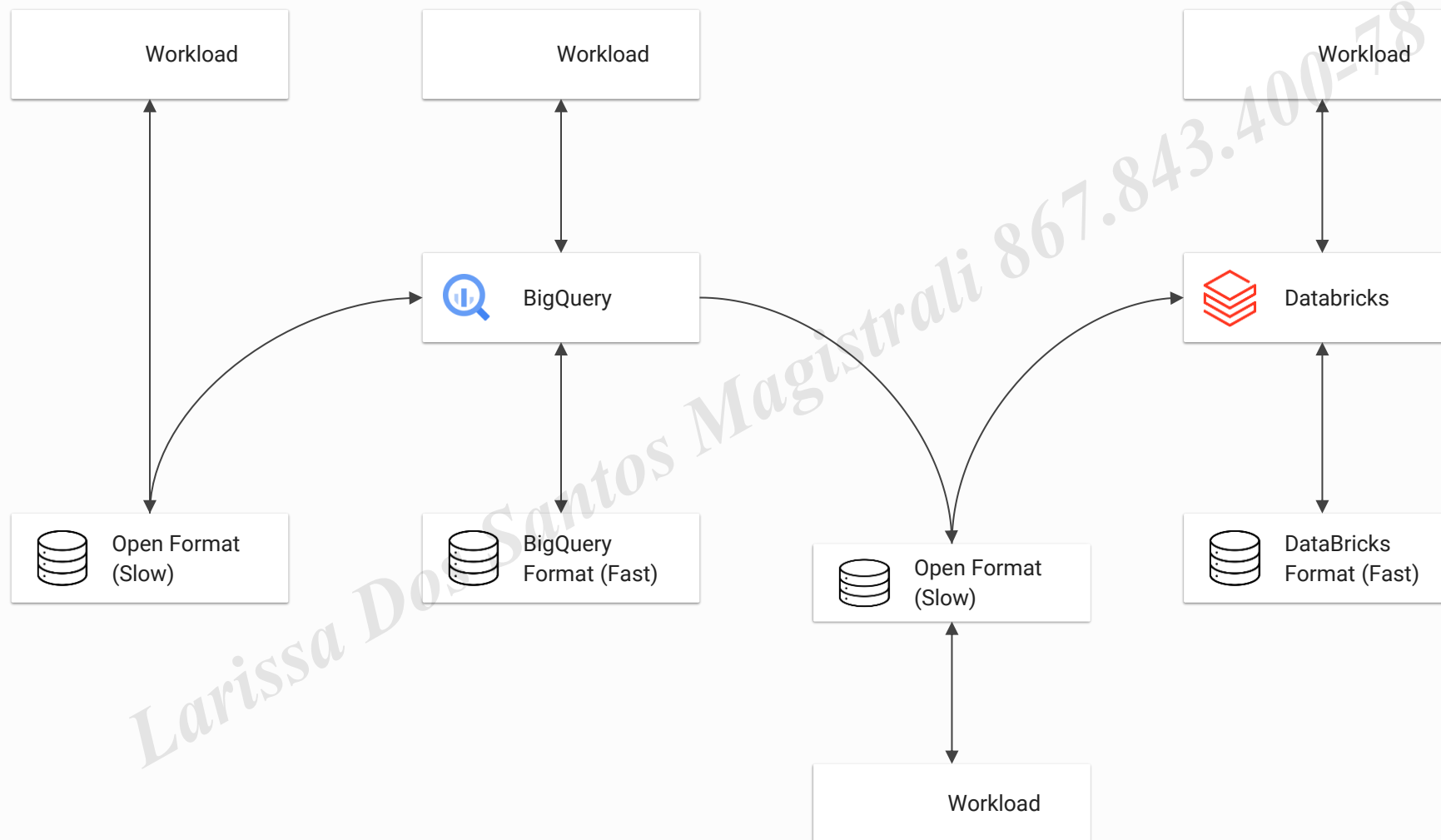
O Catálogo Iceberg - Mantém a localização atual do ponteiro de metadados atual

A Camada de Metadados - Contém os arquivos de metadados, listas de manifesto e arquivos de manifesto

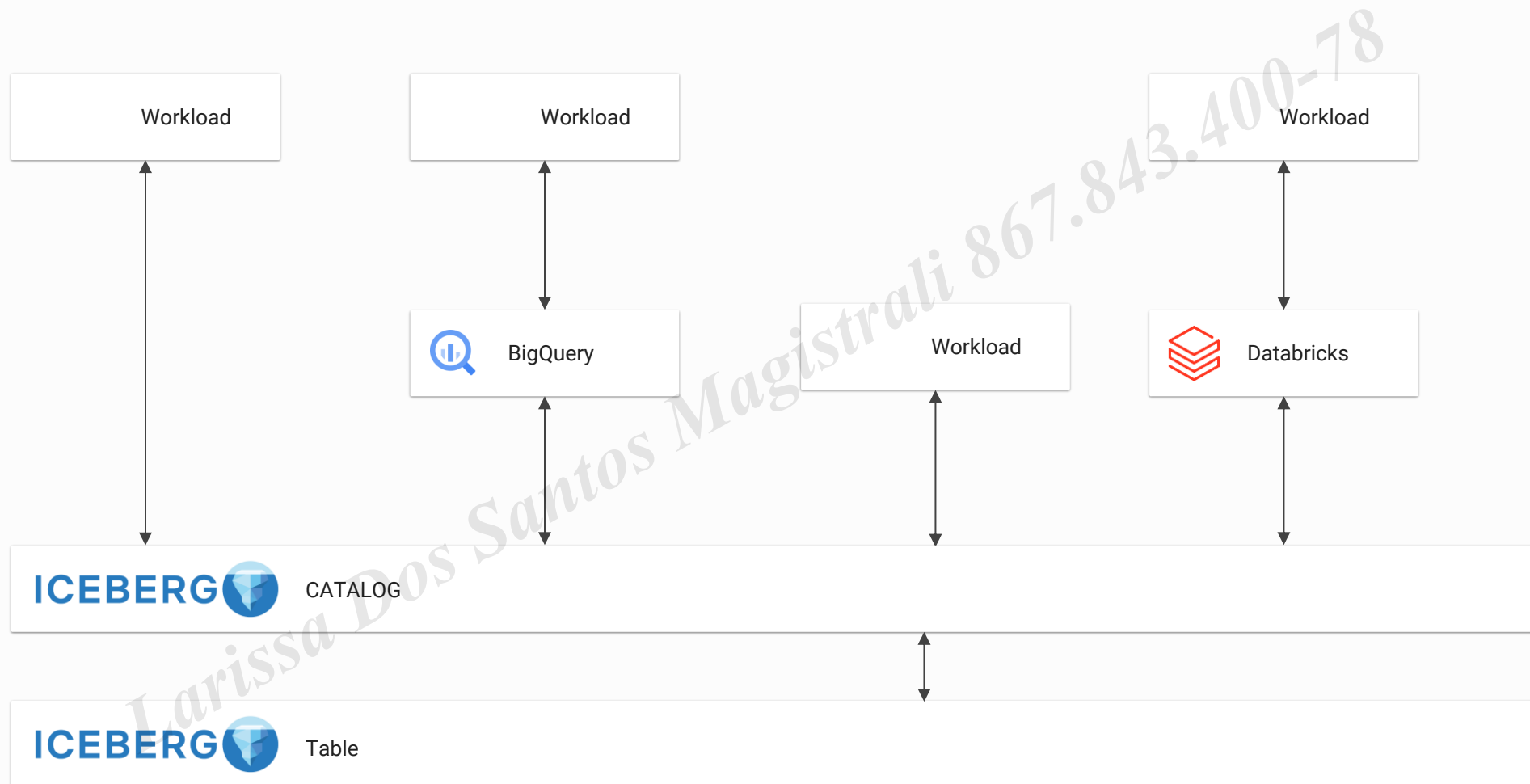
A Camada de Dados - Arquivos de dados físicos



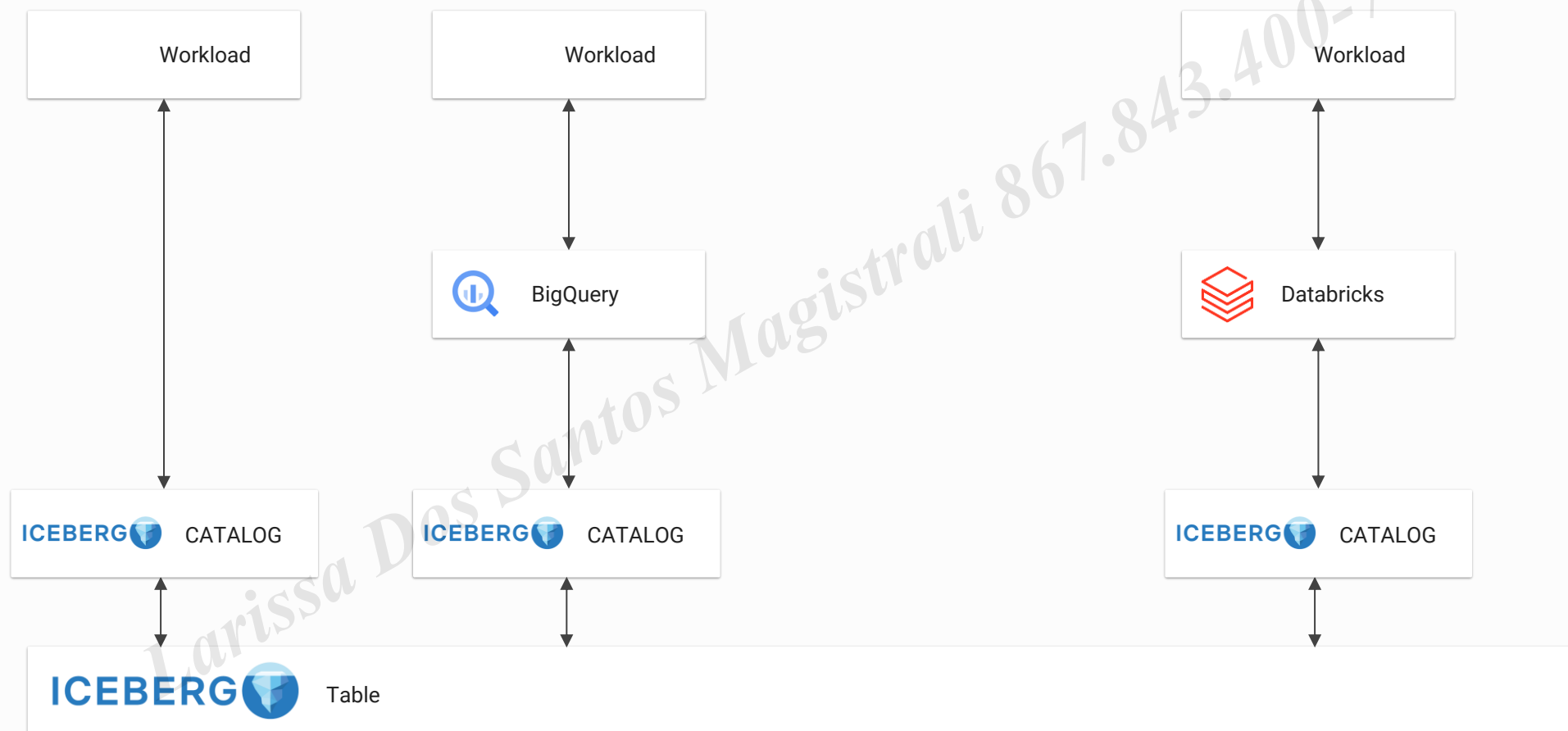
O problema que o Iceberg está tentando resolver



Iceberg: A Visão



Iceberg: A Realidade



Coletar

Processar

Armazenar

Analisar

Ativar

Empoderar

Apache SparkSQL

Larissa Dos Santos Magistrali 867.843.400-78

What is Spark SQL?

Um módulo Spark para processamento de dados estruturados.

Principais características:

- **Acesso unificado a dados:** leia e grave dados de vários formatos estruturados (como JSON, Hive, Parquet, JDBC).
- **Consultas SQL:** execute consultas SQL padrão diretamente em grandes conjuntos de dados e Spark DataFrames.
- **API DataFrame e Dataset:** Fornece uma API poderosa e de alto nível em Python, Java, Scala e R para manipulação de dados. É como uma planilha, mas distribuída entre vários computadores!
- **Otimização de desempenho:** inclui o otimizador Catalyst, que encontra automaticamente a maneira mais eficiente de executar suas consultas.

Por que usar o Spark SQL?

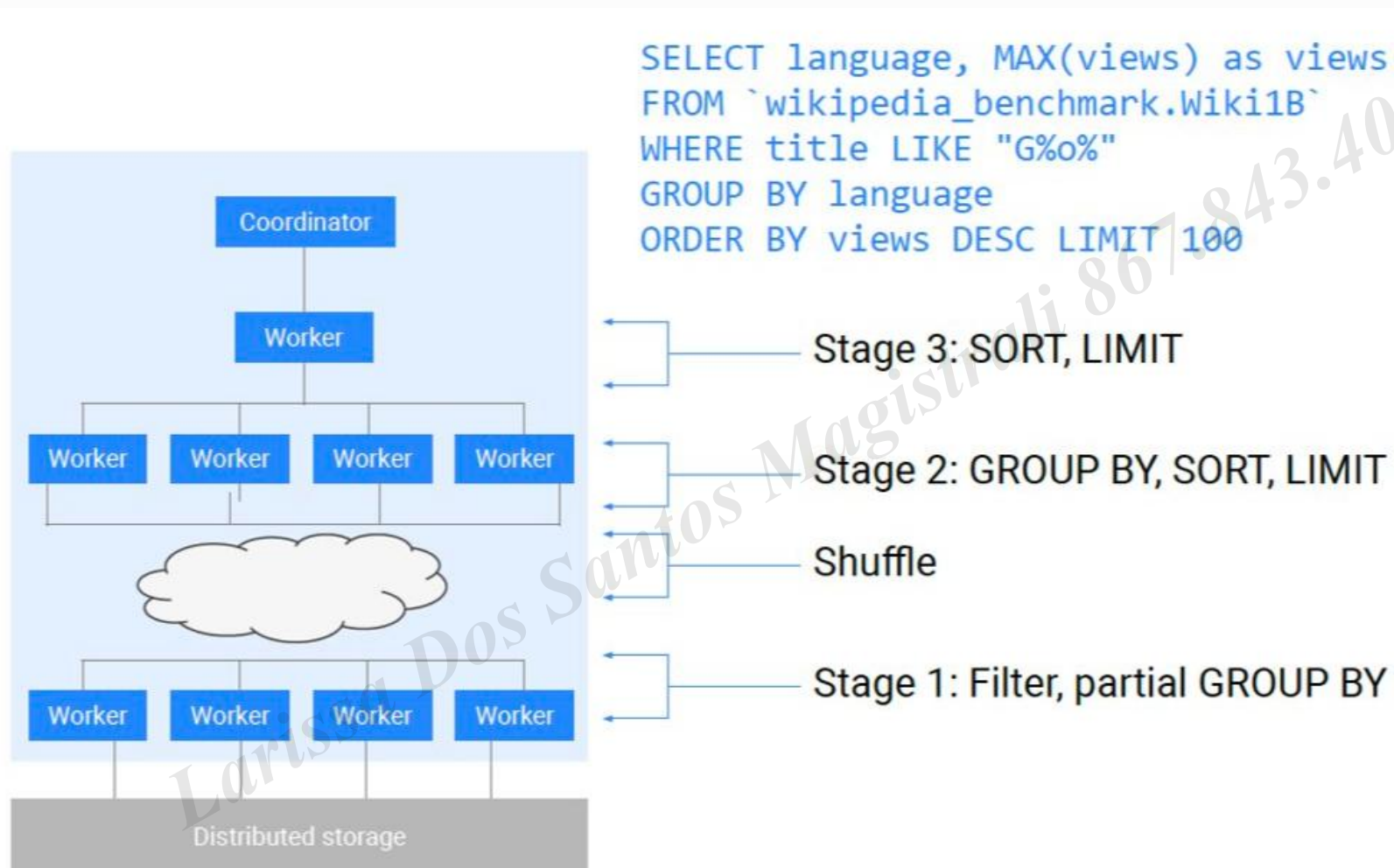
Ele combina SQL familiar com o poder da computação distribuída.

Principais benefícios:

- **Escalabilidade:** processe conjuntos de dados de qualquer tamanho, de quilobytes a petabytes, em um cluster de computadores.
- **Alto desempenho:** o otimizador Catalyst e o mecanismo de execução Tungsten compilam suas consultas em código altamente eficiente.
- **API unificada:** misture perfeitamente consultas SQL com a API programática DataFrame em Python, Scala ou Java no mesmo aplicativo.
 - `df.filter(df['age'] > 21)` é o mesmo que `spark.sql("SELECT * FROM table WHERE age > 21")`
- **Conectividade:** conecta-se a uma ampla variedade de fontes de dados imediatamente, incluindo bancos de dados NoSQL, sistemas de arquivos e data warehouses.

É o padrão para análises de dados modernas e em larga escala!

Plano de execução de consulta do BigQuery



<https://www.flickr.com/photos/197034819@N07/53080829473/>

SparkSQL

```
spark = SparkSession.builder \  
    .appName("IcebergIntegrationApp") \  
    .config("spark.sql.extensions", "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions") \  
    .config("spark.sql.catalog.local", "org.apache.iceberg.spark.SparkCatalog") \  
    .config("spark.sql.catalog.local.type", "hadoop") \  
    .config("spark.sql.catalog.local.warehouse", "file:///tmp/iceberg_warehouse") \  
    .getOrCreate()  
  
spark.sql("CREATE NAMESPACE IF NOT EXISTS local.nyc;")  
print("Namespace 'local.nyc' created or already exists.")  
  
spark.sql("""  
    CREATE TABLE IF NOT EXISTS local.nyc.taxis (  
        vendor_id BIGINT,  
        trip_id BIGINT,  
        trip_distance FLOAT,  
        fare_amount DOUBLE,  
        store_and_fwd_flag STRING  
    )  
    USING iceberg  
    PARTITIONED BY (vendor_id);  
""")
```

```
spark.sql("""
    INSERT INTO local.nyc.taxis VALUES
    (1, 1000371, 1.8, 15.32, 'N'),
    (2, 1000372, 2.5, 22.15, 'N'),
    (2, 1000373, 0.9, 9.80, 'N'),
    (1, 1000374, 4.2, 33.40, 'Y');
""")
print("Data inserted into 'local.nyc.taxis'.")

# -----
# 4. Run a SELECT Statement to Query the Data
# -----
# Read the data back from the Iceberg table.
print("\n--- Querying all data from the table ---")
results_df = spark.sql("SELECT * FROM local.nyc.taxis;")

# Show the query results
results_df.show()
```

Plano de Execução de Consulta

No mundo do poderoso mecanismo SQL do Apache Spark, cada consulta que você escreve embarca em uma jornada de transformação antes de poder gerar resultados. Essa jornada é orquestrada por meio de uma série de planos, principalmente o **plano lógico** e o **plano físico**.

- Plano Lógico: Pense no plano lógico como o "o quê" da sua consulta, uma representação abstrata de alto nível das manipulações de dados desejadas.
- Plano Físico: O plano físico, por outro lado, é o "como", um projeto detalhado e executável que determina as etapas precisas que o Spark executará em seu cluster distribuído para realizar o trabalho.

Physical Plan

-- Default Output

```
EXPLAIN select k, sum(v) from values (1, 2), (1, 3) t(k, v) group by k;
```

```
+-----+
```

```
|                                plan|
```

```
+-----+
```

```
| == Physical Plan ==
```

```
*(2) HashAggregate(keys=[k#33], functions=[sum(cast(v#34 as bigint))])
```

```
+ Exchange hashpartitioning(k#33, 200), true, [id=#59]
```

```
  +- *(1) HashAggregate(keys=[k#33], functions=[partial_sum(cast(v#34 as bigint))])
```

```
    +- *(1) LocalTableScan [k#33, v#34]
```

```
|
```

```
+-----+
```

Obrigado!

Rodrigo Vale | [linkedin.com/in/rodrigovale](https://www.linkedin.com/in/rodrigovale)

MBAUSP
ESALQ